

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Утверждена распоряжением по институту

Допущен к защите

о

т

г

№

**И.о. директора департамента цифровых,
робототехнических систем и электроники**

Азаров Иван Валерьевич

кандидат экономических наук, доцент

-Р-12.00

« ____ » _____ 20 ____ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Разработка информационной системы управления доставкой БПЛА

Выполнил:

Юрьев Илья Евгеньевич

с

Направления подготовки 09.03.04

Программная инженерия, направленность

(профиль) «Разработка и сопровождение

программного обеспечения», очной

формы обучения

т

4

(подпись)

Руководитель:

Новикова Елена Николаевна

кандидат физико-математических наук,

доцент, доцент департамента цифровых,

робототехнических систем и электроники

Нормоконтролер:

Орлова Анна Юрьевна

кандидат экономических наук, доцент, доцент

департамента цифровых, робототехнических

систем и электроники

(подпись)

(подпись)

Дата защиты

« ____ » _____ 20 ____ г.

Оценка _____

Ставрополь, 2026 г.

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

«УТВЕРЖДАЮ»

И.о. директора департамента цифровых,
робототехнических систем и электроники

И. В. Азаров

подпись, инициалы, фамилия

" ____ " ____ 20 ____ г

**ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(ДИПЛОМНЫЙ ПРОЕКТ)**

Студент Юрьев Илья Евгеньевич группа ПИЖ-б-о-22-1

1. Тема Разработка информационной системы управления доставкой БПЛА

Утверждена распоряжением по институту № 07.2-Р-12.00 от "26" января 2026 г.

2. Срок представления работы к защите «08» июня 2026 г.

Исходные данные для проектирования Требования к ВКР и порядку их выполнения

4. Содержание пояснительной записки:

Аналитический раздел

Проектный раздел

Экономический раздел

Безопасность и экологичность проекта

4.5 Другие разделы проекта _____

5 Перечень графического материала (с точным указанием обязательных чертежей) _____

Дата выдачи задания _____

Руководитель проекта _____

подпись

Е.Н. Новикова

инициалы, фамилия

Консультанты по разделам:

Аналитический раздел _____

подпись

Е.Н. Новикова

инициалы, фамилия

Проектный раздел _____

подпись

Е.Н.Новикова

инициалы, фамилия

Экономический раздел _____

подпись

А.Ю. Орлова

инициалы, фамилия

Безопасность и экологичность _____

подпись

Е.Н. Новикова

инициалы, фамилия

Нормоконтроль _____

подпись

А.Ю.Орлова

инициалы, фамилия

Задание принял к исполнению

" ____ " ____

20 ____ г.

подпись

**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

КАЛЕНДАРНЫЙ ПЛАН

Фамилия, имя, отчество (полностью) Юрьев Илья Евгеньевич

Тема ВКР «Разработка информационной системы управления доставкой БПЛА».

Руководитель: Новикова Елена Николаевна

Консультанты: Е.Н. Новикова, А.Ю. Орлова

№	Наименование этапов выпускной квалификационной работы	Срок выполнения работы	Примечание
	1. Выдача задания	26.01.2026	
	2. Начало проектирования	27.01.2026	
	3. Разделы ВКР	27.01.2026-06.06.2026	
	3.1. Аналитический раздел	27.01.2026-15.02.2026	
	3.2. Проектный раздел	16.02.2026-24.05.2026	
	3.3. Экономический раздел	25.05.2026-30.05.2026	
	3.4 Безопасность и экологичность проекта	01.06.2026-06.06.2026	
	4. Предзащита	08.06.2026-10.06.2026	
	5. Рецензирование, нормоконтроль, проверка в системе антиплагиат	11.06.2026-13.06.2026	
	6. Ознакомление с отзывом	15.06.2026-16.06.2026	
	7. Сдача ВКР, отзыва в ГЭК	17.06.2026-20.06.2026	
	8. Защита в ГЭК	25-26.06.2026	

Научный руководитель _____ Новикова Елена Николаевна
подпись, Ф.И.О.

И.о. директора департамента цифровых,
робототехнических систем и электроники _____ Азаров Иван Валерьевич
подпись, Ф.И.О.

" ____ " _____ 20 ____ г.

СОДЕРЖАНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ СОКРАЩЕНИЙ	10
ВВЕДЕНИЕ.....	12
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ	14
1.1. Характеристика предметной области	14
1.1.1. Общая характеристика и основные понятия	14
1.1.2. Анализ типовых сценариев деятельности целевой аудитории	15
1.1.3. Нормативная база.....	17
1.1.4. Выводы по результатам анализа предметной области.....	18
1.2. Исследование существующих программных решений.....	18
1.2.1. Критерии сравнения программных продуктов	18
1.2.2. Анализ аналога № 1 – Zipline.....	18
1.2.3. Анализ аналога № 2 – Wing	18
1.2.4. Анализ аналога № 3 – Matternet и DJI FlyCart	19
1.2.3. Сравнительная характеристика и недостатки решений	19
1.2.4. Обоснование конкурентных преимуществ.....	19
1.3. Выявление проблем и постановка задачи на разработку.....	20
1.3.1. Формулировка проблемной ситуации.....	20
1.3.2. Цель и задачи создания программного продукта	20
1.3.3. Описание концепции будущего решения (TO-BE)	20
1.3.4. Границы проекта и ограничения	20
1.4. Моделирование и спецификация требований	21
1.4.1. Акторы и диаграмма вариантов использования	21

1.4.2. Функциональные требования.....	21
1.4.3. Нефункциональные требования	23
1.4.3.1. Требования к производительности.....	23
1.4.3.2. Требования к безопасности и защите данных.....	24
1.4.3.3. Требования к надёжности и доступности.....	24
1.4.3.4. Требования к интерфейсу и технологическому стеку.....	24
1.5. Моделирование предметной области (концептуальная модель данных)	25
1.6. Функциональное моделирование процессов.....	26
Выводы по главе 1	27
ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	28
2.1. Выбор и обоснование архитектурного стиля.....	28
2.2. Моделирование архитектуры с использованием подхода C4	29
2.2.1. Диаграмма контекста системы (C1)	29
2.2.2. Диаграмма контейнеров (C2).....	30
2.2.3. Компоненты сервера MavixServer (C3).....	30
2.2.4. Компоненты веб-кабинета MavixWeb (C3)	32
2.3. Проектирование базы данных.....	33
2.3.1. Концептуальная и логическая модели данных	33
2.3.2. Физическая модель данных и выбор СУБД	34
2.4. Проектирование пользовательского интерфейса.....	35
2.5. Детальное проектирование (уровень кода)	36
2.5.1. Диаграммы классов ключевых компонентов	36
2.5.2. Диаграммы последовательности для основных сценариев	37
2.5.3. Применение шаблонов проектирования.....	38

2.6. Обоснование выбора стека технологий	39
Выводы по главе 2.....	40
ГЛАВА 3. КОНСТРУИРОВАНИЕ (РЕАЛИЗАЦИЯ) ПРОГРАММНОГО ПРОДУКТА	41
3.1. Организация процесса разработки	41
3.1.1. Методология разработки	41
3.1.2. Система контроля версий и стратегия ветвления	41
3.2. Соответствие реализации спроектированной архитектуре	42
3.2.1. Структура проекта.....	42
3.2.2. Конфигурация приложения.....	42
3.3. Детализация ключевых компонентов (уровень C4 – Code).....	42
3.3.1. Компонент управления заявками	42
3.3.2. Компонент сигналинга WebRTC	43
3.3.3. Компонент доступа к данным.....	44
3.3.4. Компонент управления учётными записями.....	44
3.3.5. Реализация слоя сигналинга.....	44
3.3.6. Учёт ресурсов и ведение журнала	44
3.4. Реализация клиентской части (веб-кабинет администратора)	45
3.4.1. Серверное приложение и маршрутизация.....	45
3.4.2. Организация клиентского кода.....	46
3.4.3. Реализация экранных форм.....	46
3.5. Демонстрация применения принципов качественного кода	48
3.5.1. Принципы SOLID	48
3.5.2. Применение шаблонов проектирования	49
3.5.3. Принципы чистого кода	49

3.6. Интеграция с внешними сервисами и библиотеками.....	49
3.7. Обеспечение безопасности и обработка ошибок.....	50
3.7.1. Аутентификация и авторизация.....	50
3.7.2. Валидация входных данных.....	51
3.7.3. Логирование и мониторинг	51
3.7.4. Обработка исключительных ситуаций	51
3.8. Версионность и управление конфигурацией	52
Выводы по главе 3.....	52
ГЛАВА 4. ТЕСТИРОВАНИЕ, ОБЕСПЕЧЕНИЕ КАЧЕСТВА И СОПРОВОЖДЕНИЕ ПРОГРАММНОГО ПРОДУКТА	53
4.1. Стратегия тестирования	53
4.2. Модульное тестирование.....	54
4.3. Интеграционное и системное тестирование.....	56
4.3.3. Тестирование безопасности	57
4.3.4. Регрессионное тестирование.....	57
4.4. Нагрузочное тестирование	57
4.5. Результаты тестирования и анализ дефектов	58
4.6. Подготовка к сопровождению	59
Выводы по главе 4.....	60
ГЛАВА 5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА...	61
5.1. Планирование ресурсов и расчёт трудоёмкости.....	61
5.2. Расчёт себестоимости разработки	61
5.3. Оценка экономической эффективности.....	62
5.4. Основные технико-экономические показатели	63
5.5. Обоснование экономической целесообразности	64

Выводы по главе 5	65
ГЛАВА 6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА ПРИ ЭКСПЛУАТАЦИИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	66
6.1. Анализ опасных и вредных факторов на рабочем месте	66
6.2. Мероприятия по обеспечению безопасности	67
6.3. Электробезопасность и пожарная безопасность	69
Выводы по главе 6	69
ЗАКЛЮЧЕНИЕ	71
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	73
ПРИЛОЖЕНИЕ Г	76
ПРИЛОЖЕНИЕ Д	80
Д.1. Развёртывание и требования к окружению	80
Д.2. Регистрация и вход в систему	80
Д.3. Управление операторами и дронами	80
Д.4. Создание заявки и журнал доставок	81
Д.5. Загрузка дистрибутивов клиентов	81
Д.6. Сообщения об ошибках и нештатные ситуации	81
ПРИЛОЖЕНИЕ Е	82

СПИСОК ИСПОЛЬЗОВАННЫХ СОКРАЩЕНИЙ

- БД – база данных;
- БПЛА – беспилотный летательный аппарат;
- ИС – информационная система;
- ПО – программное обеспечение;
- СУБД – система управления базами данных;
- ЗНФ – третья нормальная форма;
- API (Application Programming Interface) – программный интерфейс приложения;
- С4 – модель визуализации архитектуры (контекст, контейнеры, компоненты, код);
- CI/CD – непрерывная интеграция и непрерывная поставка;
- CORS (Cross-Origin Resource Sharing) – разделение ресурсов между разными источниками;
- CSS (Cascading Style Sheets) – каскадные таблицы стилей;
- DFD (Data Flow Diagram) – диаграмма потоков данных;
- DI (Dependency Injection) – внедрение зависимостей;
- DOM (Document Object Model) – объектная модель документа;
- DRY / KISS / YAGNI – принципы чистого кода;
- ER (Entity-Relationship) – модель «сущность – связь»;
- HTML (HyperText Markup Language) – язык разметки гипертекста;
- HTTP / HTTPS – протокол передачи гипертекста и его защищённая версия;
- ICE / STUN / TURN – протокол и серверы установления соединения через NAT;
- IDEF0 – нотация функционального моделирования процессов;
- JWT (JSON Web Token) – формат токена авторизации;
- NAT (Network Address Translation) – преобразование сетевых адресов;

- NPV / PI – чистый дисконтированный доход и индекс доходности;
- ORM (Object-Relational Mapping) – объектно-реляционное отображение;
- P2P (peer-to-peer) – передача данных напрямую между двумя узлами;
- REST (Representational State Transfer) – архитектурный стиль веб-интерфейсов;
- SMTP (Simple Mail Transfer Protocol) – протокол передачи электронной почты;
- SOLID – пять принципов объектно-ориентированного проектирования;
- SQL (Structured Query Language) – язык структурированных запросов;
- TLS (Transport Layer Security) – протокол защиты транспортного уровня;
- WebRTC (Web Real-Time Communication) – передача медиа и данных P2P;
- WebSocket (WS) – протокол двунаправленной связи поверх HTTP.

ВВЕДЕНИЕ

Скорость доставки в ряде сфер деятельности определяет конечный результат. В медицине биоматериал, компоненты крови и лекарственные препараты нередко имеют жёсткие ограничения по времени транспортировки, и задержка наземного курьера в городском трафике напрямую влияет на качество помощи. Беспилотные летательные аппараты (БПЛА) позволяют доставлять малогабаритные грузы по прямой, минуя дорожную сеть, однако сам по себе летательный аппарат – лишь часть решения. Для практической эксплуатации флота дронов необходима информационная система, которая ведёт учёт операторов и аппаратов, формирует и распределяет заявки на доставку, фиксирует ход и результат каждого рейса, а также обеспечивает связь между наземной станцией оператора и бортом.

Настоящая работа посвящена разработке такой информационной системы управления доставкой БПЛА. Работа является подтемой комплексной выпускной квалификационной работы: аппаратно-программный комплекс самого летательного аппарата и станции оператора разрабатывается в смежной подтеме (Соколов М.Р.), тогда как в данной работе рассматривается серверная информационная система и веб-интерфейс администратора – компоненты MavixServer и MavixWeb программного комплекса Mavix.

Актуальность темы определяется ростом спроса на быструю доставку малых грузов P2P и одновременной недоступностью открытых и недорогих систем управления для парков беспилотников. Существующие промышленные платформы закрыты, дороги и привязаны к фирменному оборудованию, что ограничивает их применение в небольших организациях – медицинских лабораториях, экстренных и специальных службах.

Цель работы – автоматизировать процесс срочной доставки малогабаритных грузов с помощью управляемого через интернет БПЛА за счёт разработки информационной системы, которая обеспечивает учёт операторов и

аппаратов, ведение жизненного цикла заявки на доставку и сигнальное взаимодействие между оператором и бортом.

Для достижения цели поставлены следующие задачи:

- провести анализ предметной области и существующих решений, сформировать требования к информационной системе;
- спроектировать архитектуру системы, модель данных и программные интерфейсы;
- обоснованно выбрать стек технологий и шаблоны проектирования;
- реализовать серверную часть и веб-кабинет администратора;
- провести модульное, интеграционное и нагрузочное тестирование, оценить покрытие кода;
- выполнить технико-экономическое обоснование и рассмотреть вопросы охраны труда при эксплуатации системы.

Объектом исследования является процесс управления доставкой малогабаритных грузов с использованием беспилотных летательных аппаратов. Предмет исследования – методы и средства построения информационной системы, автоматизирующей этот процесс.

Практическая значимость работы состоит в том, что разработанная система образует управляющий контур для парка БПЛА на открытом стеке и серийном оборудовании и пригодна для внедрения в организациях со срочной логистикой малых грузов.

Записка включает введение, шесть разделов (анализ, проектирование, реализация, тестирование, технико-экономическое обоснование, охрана труда), заключение, список источников и приложения.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

1.1. Характеристика предметной области

1.1.1. Общая характеристика и основные понятия

Предметной областью работы является срочная логистика малогабаритных грузов с применением беспилотных летательных аппаратов и, в более узком смысле, управление парком таких аппаратов и потоком заявок на доставку. Под малогабаритным грузом понимается отправление массой до трёх килограммов, для которого критично время доставки: медицинские пробы и биоматериалы, компоненты крови, вакцины, лекарственные препараты экстренного применения, небольшие документы и комплектующие.

Особенностью предметной области является сочетание двух разнородных задач – управления физическим объектом (летательным аппаратом) и управления информационным процессом (учёт, распределением и контролем заявок). Настоящая работа сосредоточена на второй задаче: на программных средствах, которые образуют управляющий и учётный контур для парка беспилотников. Такой контур характерен для класса информационных систем оперативного управления, к которому относятся системы диспетчеризации транспорта, службы доставки и геоинформационные сервисы; общими их чертами являются ведение справочников ресурсов, обработка потока заявок с переходом по состояниям и фиксация истории операций.

Грузы, для которых применима срочная воздушная доставка, классифицируются по требованиям к условиям транспортировки. Медицинские грузы (пробы, кровь, вакцины) требуют минимального времени в пути и соблюдения температурного режима. Документы и комплектующие менее чувствительны ко времени, но также выигрывают от прямого маршрута. Общим для всех видов является малая масса и высокая ценность оперативности доставки, что и определяет целесообразность применения беспилотников на коротких расстояниях.

Ключевые понятия предметной области приведены ниже. Беспилотный летательный аппарат (БПЛА, дрон) – летательный аппарат без экипажа на борту, управляемый дистанционно. Оператор – лицо, которое принимает заявку и дистанционно ведёт аппарат. Администратор – лицо, управляющее парком дронов, учётными записями операторов и потоком заявок. Заявка на доставку – запись о необходимости переместить груз из точки отправления в точку назначения. Информационная система управления доставкой – программное обеспечение, автоматизирующее учёт участников, аппаратов и заявок, а также организующее связь между наземной станцией и бортом.

Разрабатываемая информационная система занимает место управляющего и учётного контура: она не управляет полётом напрямую (это задача станции оператора и бортового модуля), но определяет, какой оператор какой груз и каким аппаратом доставляет, фиксирует результат и обеспечивает начальную установку соединения между оператором и бортом.

Рынок доставки беспилотниками демонстрирует устойчивый рост: развиваются проекты доставки медицинских грузов, посылок и продуктов, совершенствуется нормативная база, снижается стоимость аппаратов и полётных контроллеров. Вместе с тем управляющее программное обеспечение остаётся преимущественно закрытым и предлагается в составе дорогих платформ. Наблюдается тенденция к разделению аппаратной и программной частей: летательный аппарат строится из серийных компонентов, а управление и учёт переносятся в веб-ориентированные информационные системы. Это создаёт спрос на открытые, переносимые и недорогие решения, к которым относится разрабатываемая система.

1.1.2. Анализ типовых сценариев деятельности целевой аудитории

Целевая аудитория системы – организации, для которых характерна срочная доставка малых грузов на короткие расстояния (до 15–20 км). Выделены три сегмента: медицинские лаборатории и клиники; экстренные и специальные службы; небольшие логистические операторы. Для таких организаций типична

территориально распределённая структура – центральная площадка и несколько удалённых подразделений, между которыми регулярно перемещаются пробы, образцы и расходные материалы.

В текущем процессе (AS-IS) пробы между точками возит штатный курьер. В часы пик дорога занимает продолжительное время, заявки накапливаются быстрее, чем курьер успевает их обрабатывать, образуется очередь, и часть проб теряет годность. Учёт заявок ведётся в журнале и мессенджерах, что приводит к потере информации и отсутствию прозрачности.

В целевом состоянии (TO-BE) процесс перестраивается за счёт внедрения комплекса беспилотных летательных аппаратов и информационной системы управления доставкой. Администратор формирует заявку в системе, указывая точку назначения; заявка автоматически предлагается свободным операторам и закрепляется за тем, кто принял её первым. После закрепления оператор устанавливает соединение с бортом, а беспилотный аппарат выполняет доставку по прямому маршруту. На всех этапах система фиксирует статусы и ведёт журнал, обеспечивая прозрачность и предсказуемость сроков. Контекстная модель целевого процесса в нотации IDEF0 (уровень A-0) приведена на рисунке 1.1: процесс доставки управляется информационной системой Mavix, выступающей одним из механизмов его выполнения.



Рисунок 1.1 – Контекстная диаграмма целевого процесса доставки (IDEF0, уровень А-0, состояние ТО-ВЕ)

Анализ существующего и целевого процессов позволяет выделить основные проблемы целевой аудитории: непредсказуемое время доставки, отсутствие единого учёта и журнала, невозможность контролировать ход доставки в реальном времени и высокую стоимость содержания курьерской службы. Перечисленные проблемы определяют состав функций разрабатываемой системы.

На основе анализа определены основные сценарии использования системы её участниками. Администратор создаёт заявку на доставку, указывая точку назначения на карте, после чего заявка автоматически предлагается свободным операторам, что исключает необходимость телефонных звонков и согласований. Оператор просматривает перечень предложенных заявок и принимает одну из них; закрепление выполняется по принципу приоритета первого принявшего, после чего оператор подключается к борту для сопровождения доставки. Кроме того, администратор ведёт учёт операторов и дронов, управляя составом парка и правами доступа, а также просматривает журнал доставок со статусами для контроля исполнения и анализа результатов работы.

1.1.3. Нормативная база

Деятельность в области использования БПЛА регулируется нормами использования воздушного пространства Российской Федерации и правилами выполнения полётов беспилотных воздушных судов; персональные данные операторов обрабатываются с учётом Федерального закона № 152-ФЗ «О персональных данных». Применительно к информационной системе значимы стандарты жизненного цикла и документации ПО: ГОСТ Р ИСО/МЭК 12207 [9], ЕСПД (ГОСТ серии 19) [10], рекомендации IEEE Std 830 [12] по спецификации требований. Вопросы сертификации конкретных маршрутов полётов находятся вне границ работы.

1.1.4. Выводы по результатам анализа предметной области

Анализ показал, что срочная доставка малых грузов наземным транспортом медленна и непрозрачна, а для эксплуатации флота БПЛА необходим управляющий контур – информационная система учёта участников, аппаратов и заявок. Это определяет состав функций разрабатываемой системы.

1.2. Исследование существующих программных решений

1.2.1. Критерии сравнения программных продуктов

Для сравнения аналогов выбраны критерии: функциональные возможности управления доставкой, открытость платформы и исходного кода, привязка к фирменному оборудованию, канал связи с бортом, стоимость и модель распространения, возможность самостоятельного развёртывания.

1.2.2. Анализ аналога № 1 – Zipline

Zipline (США, Руанда) – промышленная платформа доставки медицинских грузов (кровь, вакцины) беспилотниками самолётного типа с большим радиусом действия и сбросом груза на парашюте. Платформа включает собственные аппараты, распределительные центры и закрытое программное обеспечение управления. Сильные стороны: высокая дальность, отлаженная логистика, медицинская специализация. Слабые стороны для рассматриваемого сценария: закрытость, высокая стоимость владения, ориентация на крупные инфраструктурные проекты и невозможность развёртывания силами небольшой организации. Модель распространения – поставка «под ключ» в составе инфраструктуры.

1.2.3. Анализ аналога № 2 – Wing

Wing (подразделение Alphabet) – сервис доставки лёгких посылок мультикоптерами с опусканием груза на лебёдке. Сильные стороны: городская доставка «до двери», интеграция с торговыми партнёрами. Слабые стороны: закрытая экосистема, работа только в зонах присутствия сервиса, отсутствие

возможности самостоятельного развёртывания и доработки. Технологический стек и протоколы управления не раскрываются.

1.2.4. Анализ аналога № 3 – Matternet и DJI FlyCart

Matternet специализируется на медицинской логистике между клиниками с использованием посадочных станций и закрытого ПО управления маршрутами; акцент сделан на повторяемых маршрутах P2P. DJI FlyCart 30 – тяжёлый грузовой дрон с фирменным приложением управления, ориентированный на промышленную логистику и строительство. Общими для обоих решений являются закрытость платформы, привязка к фирменному оборудованию и высокая стоимость, что ограничивает их применение в небольших организациях со срочной доставкой малых грузов.

1.2.3. Сравнительная характеристика и недостатки решений

Таблица 1.1 – Сравнительная характеристика существующих решений

Критерий	Промышленные аналоги	Mavix (разрабатываемая ИС)
Открытость платформы	закрытая, проприетарная	открытый исходный код
Привязка к оборудованию	фирменное «железо»	серийные комплектующие, типовой FC
Канал связи с бортом	своя радиосеть	интернет, WebRTC P2P
Стоимость владения	высокая	низкая
Самостоятельное развёртывание	недоступно	Docker, один сервер
Учёт и журнал доставок	есть, закрыт	есть, открытая модель данных

Сравнение показывает, что промышленные платформы превосходят разрабатываемую систему по дальности и автономности полёта, однако уступают ей в открытости, стоимости и возможности доработки под конкретного заказчика, что и определяет рыночную нишу проекта.

1.2.4. Обоснование конкурентных преимуществ

Общими недостатками аналогов являются закрытость, высокая стоимость и привязка к фирменному оборудованию. Разрабатываемая информационная система занимает свободную нишу: открытый управляющий контур, который разворачивается на одном сервере, не требует проприетарного «железа» и

использует стандартные веб-технологии. Это делает решение доступным для небольших организаций.

1.3. Выявление проблем и постановка задачи на разработку

1.3.1. Формулировка проблемной ситуации

Отсутствует доступная открытая информационная система, которая позволила бы небольшой организации вести учёт операторов и дронов, распределять заявки на срочную доставку и контролировать их исполнение, а также организовывать связь оператора с бортом без закупки дорогой проприетарной платформы.

1.3.2. Цель и задачи создания программного продукта

Цель – создать информационную систему управления доставкой БПЛА (сервер и веб-кабинет администратора), автоматизирующую учёт участников и аппаратов, полный жизненный цикл заявки и сигнальное взаимодействие оператора с бортом. Задачи перечислены во введении и детализируются в требованиях (п. 1.4).

1.3.3. Описание концепции будущего решения (ТО-ВЕ)

В целевой концепции администратор регистрируется в системе и формирует парк: создаёт учётные записи операторов и принимает самостоятельную регистрацию бортов. Поступающая заявка на доставку рассылается операторам; первый принявший оператор «захватывает» заявку и подключается к назначенному дрону. Видео, телеметрия и команды передаются напрямую между оператором и бортом, а система фиксирует смену статусов заявки и ведёт журнал.

1.3.4. Границы проекта и ограничения

В границы данной подтемы входят серверная информационная система (MavixServer) и веб-кабинет администратора (MavixWeb). Бортовой модуль и приложение оператора разрабатываются в смежной подтеме и рассматриваются здесь как внешние потребители программных интерфейсов. Технологические

ограничения: реляционная СУБД, контейнеризация, работа за NAT через STUN/TURN [24]. Ресурсные ограничения – индивидуальная разработка в сроки выполнения ВКР.

1.4. Моделирование и спецификация требований

1.4.1. Акторы и диаграмма вариантов использования

Выделены два основных актора, взаимодействующих с информационной системой: администратор и оператор. Внешними по отношению к системе являются бортовой модуль (регистрируется и обменивается сигнальными сообщениями) и внешние сервисы (почтовый шлюз, серверы STUN/TURN). Диаграмма вариантов использования приведена на рисунке 1.2.

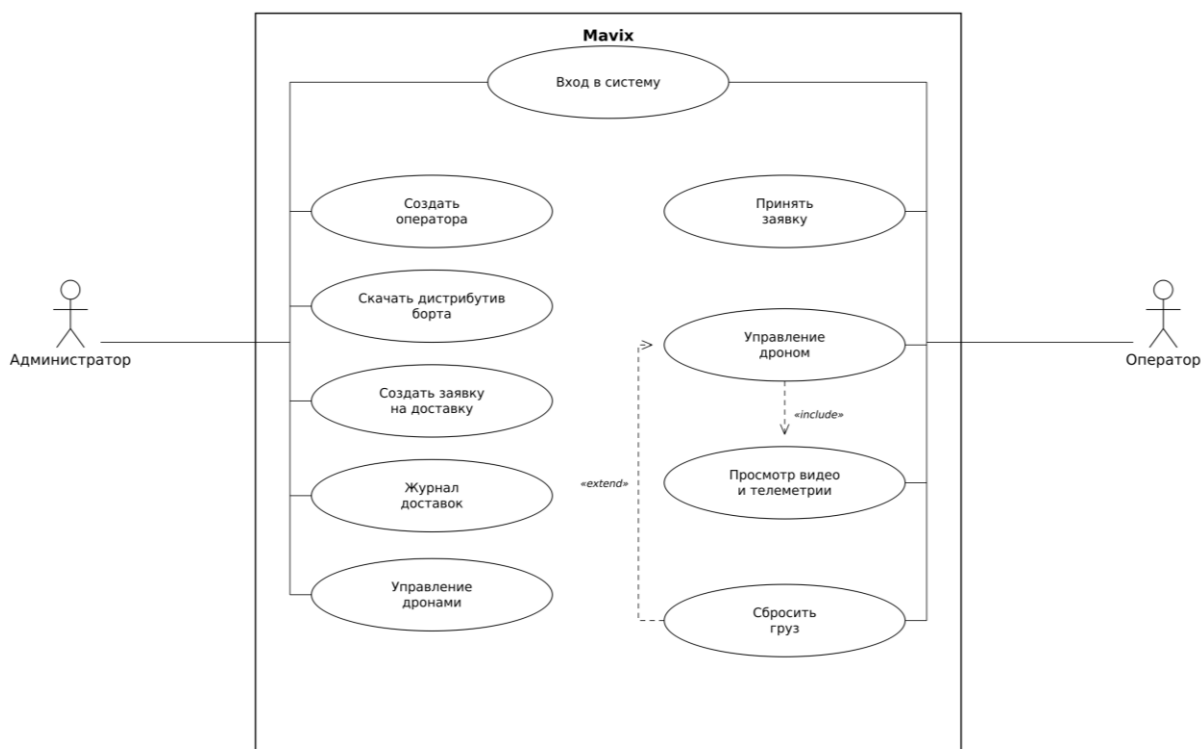


Рисунок 1.2 – Диаграмма вариантов использования

1.4.2. Функциональные требования

Функциональные требования к информационной системе сведены в таблицу 1.2 с привязкой к вариантам использования.

Таблица 1.2 – Функциональные требования к информационной системе

Код	Функциональное требование	Актор
ФТ-1	Регистрация и вход администратора (JWT), сброс пароля	Администратор
ФТ-2	Создание и блокировка учётных записей операторов	Администратор
ФТ-3	Просмотр и удаление дронов, самостоятельная регистрация борта	Администратор, борт
ФТ-4	Создание заявки на доставку (дрон, адрес/координаты)	Администратор
ФТ-5	Просмотр предложенных заявок и атомарный приём	Оператор
ФТ-6	Смена статусов заявки: in_flight, delivered, cancelled	Оператор
ФТ-7	Журнал доставок со статусами и снимками данных	Администратор
ФТ-8	Сигналинг WebRTC (offer/answer/ICE) оператор – борт	Оператор, борт
ФТ-9	Выдача ICE-серверов и дистрибутивов клиентов	Оператор, борт

Для наиболее значимых вариантов использования составлены детальные спецификации прецедентов. В таблице 1.3 приведён прецедент «Создание заявки на доставку», в таблице 1.4 – «Приём заявки оператором».

Таблица 1.3 – Спецификация прецедента «Создание заявки на доставку»

Элемент прецедента	Описание
Название	Создание заявки на доставку
Актор	Администратор
Предусловия	Администратор авторизован; в системе есть хотя бы один дрон
Основной поток	1) администратор открывает раздел «Доставки»; 2) выбирает дрон; 3) указывает адрес или точку на карте; 4) подтверждает создание; 5) система сохраняет заявку в статусе offered и рассылает её операторам
Альтернативы	Нет свободных дронов – система выводит предупреждение
Постусловия	Заявка создана и доступна операторам для приёма

Таблица 1.4 – Спецификация прецедента «Приём заявки оператором»

Элемент прецедента	Описание
Название	Приём заявки оператором
Актор	Оператор
Предусловия	Оператор авторизован; существует заявка в статусе offered
Основной поток	1) оператор видит список предложенных заявок; 2) нажимает «Принять»; 3) система атомарно закрепляет заявку за оператором (статус accepted); 4) оператор подключается к дрону
Альтернативы	Заявку уже приняли – система возвращает отказ (код 409) и обновляет список
Постусловия	Заявка закреплена ровно за одним оператором

Детальные спецификации ключевых прецедентов конкретизируют ожидаемое поведение системы и служат основой для формирования требований, проектирования интерфейсов и подготовки тест-кейсов.

1.4.3. Нефункциональные требования

Нефункциональные требования определяют качественные характеристики системы и ограничения, в рамках которых она должна функционировать. Они сгруппированы в четыре раздела по типу налагаемых ограничений: производительность, безопасность и защита данных, надёжность и доступность, а также интерфейс и технологический стек.

1.4.3.1. Требования к производительности

NFR-01. Время отклика на запросы чтения (получение списков операторов, дронов и заявок, проверка живости сервера) – не более 100 миллисекунд при типовой нагрузке.

NFR-02. Сервер должен обслуживать поток обращений от нескольких десятков одновременно работающих пользователей без отказов и заметной деградации производительности.

NFR-03. Фаза установления сигнального соединения между оператором и бортом – не более нескольких секунд; пропускная способность подсистемы сигналинга должна быть достаточной для одновременного ведения нескольких доставок.

1.4.3.2. Требования к безопасности и защите данных

NFR-04. Доступ к защищённым операциям предоставляется только аутентифицированным пользователям; аутентификация выполняется по токенам JWT [20, 21] с разграничением ролей администратора и оператора.

NFR-05. Пароли пользователей хранятся исключительно в виде криптографического хеша и не подлежат хранению или передаче в открытом виде.

NFR-06. Самостоятельная регистрация борта выполняется по отдельному enrollment-токену, не предоставляющему иных прав в системе.

NFR-07. Доступ к ресурсам разграничивается по их владельцу: администратор работает только с принадлежащими ему операторами, дронами и заявками.

NFR-08. Система должна быть защищена от внедрения SQL-кода за счёт параметризации запросов средствами ORM и от передачи некорректных данных за счёт серверной валидации; обмен данными выполняется по протоколу TLS.

1.4.3.3. Требования к надёжности и доступности

NFR-09. Журнал доставок должен сохраняться при удалении связанных дронов и операторов, что обеспечивается денормализованными снимками и действием SET NULL для внешних ключей.

NFR-10. При остановке система корректно завершает сессии, оповещая подключённых клиентов, и восстанавливает соединение после кратковременных сбоев сети.

NFR-11. Целевой уровень доступности системы в режиме эксплуатации – не ниже 99 процентов.

1.4.3.4. Требования к интерфейсу и технологическому стеку

NFR-12. Веб-кабинет администратора должен обладать понятной навигацией, поддерживать выбор точки доставки на интерактивной карте с автоматическим определением адреса, предоставлять журнал доставок с

индикацией статусов и запрашивать подтверждение потенциально необратимых действий.

NFR-13. Серверная часть реализуется на языке Python с асинхронным веб-фреймворком, данные хранятся в реляционной СУБД; система разворачивается в контейнерах, для работы за преобразованием сетевых адресов используются протоколы STUN и TURN, зависимость от проприетарного программного и аппаратного обеспечения не допускается.

1.5. Моделирование предметной области (концептуальная модель данных)

На основе анализа выделены основные сущности предметной области: администратор, оператор, дрон, заявка на доставку. Администратор владеет операторами, дронами и заявками. Заявка связана с дроном и оператором и проходит через ряд статусов. Концептуальная модель данных в нотации «сущность – связь» приведена на рисунке 1.3; она служит основой для проектирования физической базы данных в разделе 2.

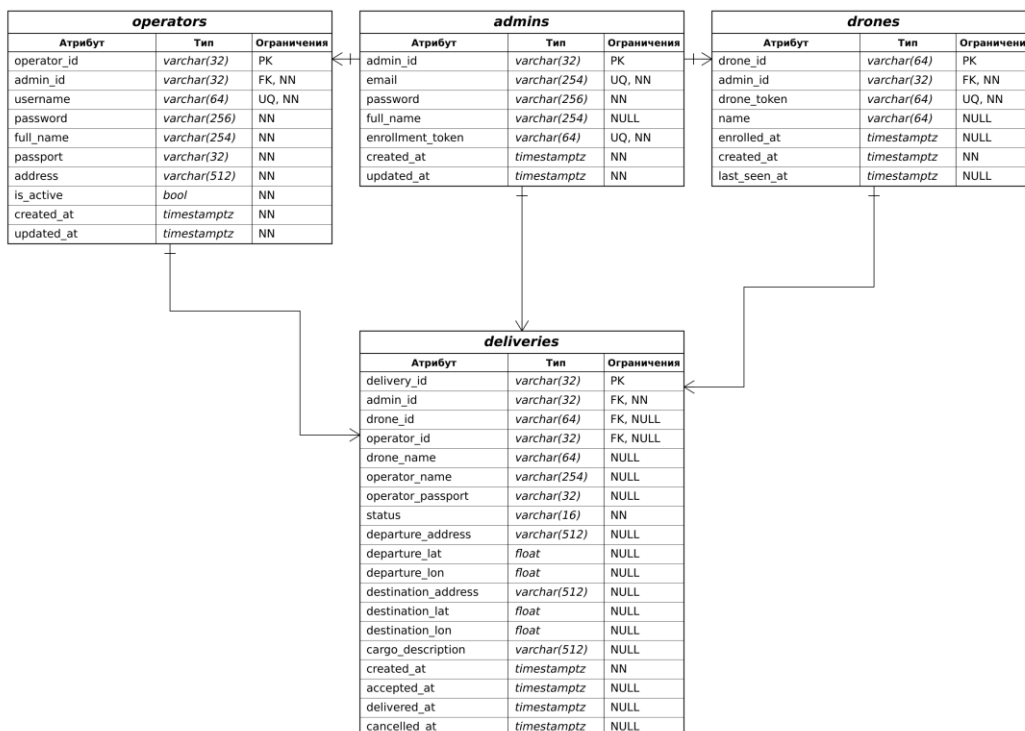


Рисунок 1.3 – Концептуальная модель данных (ER-диаграмма)

1.6. Функциональное моделирование процессов

Для представления системы как совокупности функций применена методология функционального моделирования IDEF0. Главная функция «выполнить доставку груза» получает на вход заявку, управляется регламентами и командами оператора, опирается на механизмы (дрон, станцию, информационную систему и каналы связи) и порождает на выходе доставленный груз и запись в журнале. Контекстная диаграмма функциональной модели приведена на рисунке 1.4.

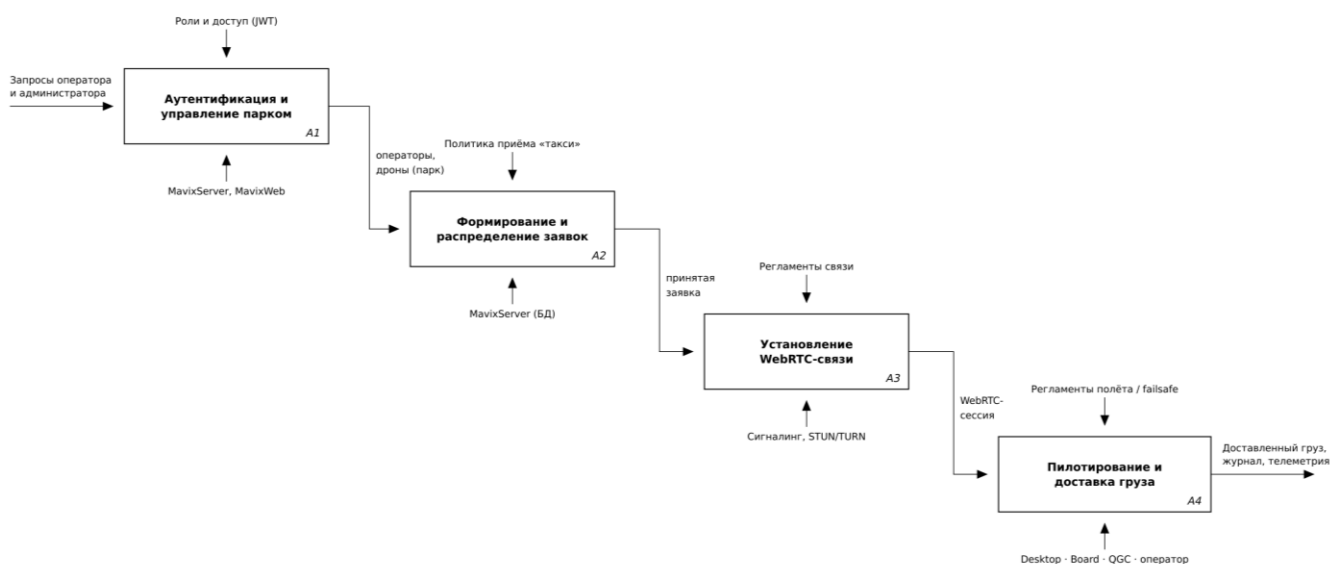


Рисунок 1.4 – Функциональная модель доставки (IDEF0)

Декомпозиция выделяет подфункции: принять и распределить заявку, установить связь оператора с бортом, выполнить доставку и сброс груза, зафиксировать результат. Информационная система отвечает за первую и четвёртую подфункции и участвует во второй (сигналинг).

Потоки данных в системе отражает диаграмма потоков данных (DFD), приведённая на рисунке 1.5. Видеопоток и телеметрия движутся от борта к оператору, команды - в обратном направлении, служебные и сигнальные

сообщения проходят через сервер, а сведения о заявках и их статусах сохраняются в хранилище данных.

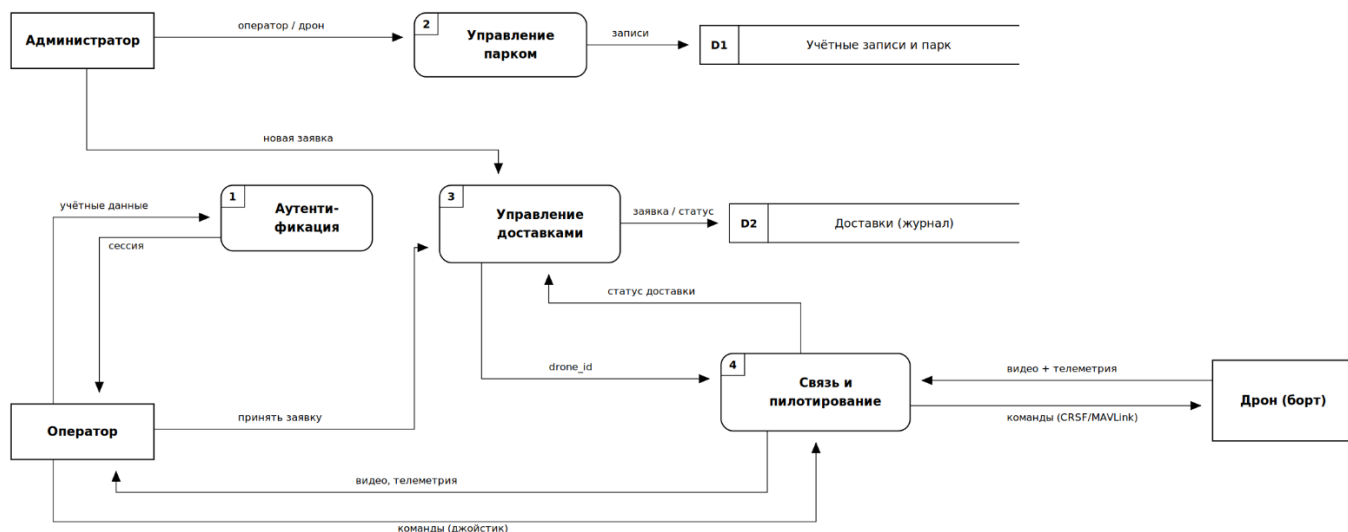


Рисунок 1.5 – Диаграмма потоков данных (DFD)

Выводы по главе 1

В результате выполнения первого раздела проведён комплексный анализ предметной области срочной доставки малогабаритных грузов с использованием БПЛА. Выявлено, что наземная доставка медленна и непрозрачна, а существующие промышленные платформы (Zipline, Wing, Matternet, DJI FlyCart) закрыты, дороги и привязаны к фирменному оборудованию.

Обоснована актуальность создания открытой информационной системы управления доставкой. Сформированы и задокументированы функциональные (девять требований) и нефункциональные требования, построена диаграмма вариантов использования с акторами «администратор» и «оператор», разработана концептуальная модель данных с сущностями «администратор», «оператор», «дрон», «заявка». Полученные результаты служат достаточной основой для перехода к проектированию архитектуры.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1. Выбор и обоснование архитектурного стиля

На основе требований раздела 1 проведён анализ архитектурных альтернатив для информационной системы. Рассмотрены монолитная, многоуровневая клиент-серверная и микросервисная архитектуры. Микросервисная архитектура обеспечивает независимое масштабирование [8] и развёртывание, однако вносит накладные расходы на межсервисное взаимодействие, согласованность данных и эксплуатацию, неоправданные при индивидуальной разработке и одном домене. Чистый монолит без внутренней структуры затрудняет сопровождение.

Выбран модульный монолит с многоуровневой организацией серверной части: один развёртываемый процесс, разделённый на слои представления (REST-роутеры) [14, 15], бизнес-логики (сервисы), доступа к данным (репозитории) и модели. Такой стиль при ожидаемой нагрузке (десятки одновременных пользователей) и единственном домене проще, дешевле и надёжнее микросервисов, но за счёт слоистости сохраняет тестируемость и возможность дальнейшего выделения сервисов.

Дополнительно применены два архитектурных решения, продиктованных характером задачи. Во-первых, передача медиа и команд между оператором и бортом построена по принципу P2P на технологии WebRTC [22, 23]: видеопоток и команды идут напрямую, минуя сервер, что минимизирует задержку и нагрузку на сервер. Во-вторых, установление соединения и оповещения о заявках реализованы как событийное взаимодействие (event-driven) по протоколу WebSocket: стороны обмениваются асинхронными сообщениями и получают уведомления без опроса. Таким образом, сервер выступает в роли посредника-сигнализатора, а не транзитного узла для медиапотока.

Сводный анализ рассмотренных архитектурных альтернатив по ключевым критериям приведён в таблице 2.1.

Таблица 2.1 – Сравнение архитектурных стилей

Критерий	Монолит	Микросервисы	Модульный монолит (выбор)
Сложность разработки	низкая	высокая	средняя
Накладные расходы	отсутствуют	высокие (сеть, согласованность)	отсутствуют
Масштабируемость	ограниченная	высокая	достаточная для задачи
Сопровождаемость	низкая	высокая	высокая (слои)
Пригодность для команды из 1 чел.	да	нет	да

Выбор модульного монолита обеспечивает баланс простоты и сопровождаемости при ожидаемой нагрузке и единственном домене, сохраняя возможность последующего выделения сервисов при росте требований.

2.2. Моделирование архитектуры с использованием подхода C4

2.2.1. Диаграмма контекста системы (C1)

На верхнем уровне система Mavix рассматривается как единое целое во взаимодействии с акторами (администратор, оператор) и внешними сервисами (почтовый шлюз для уведомлений, серверы STUN/TURN для обхода NAT, картографический сервис). Контекстная диаграмма приведена на рисунке 2.1.

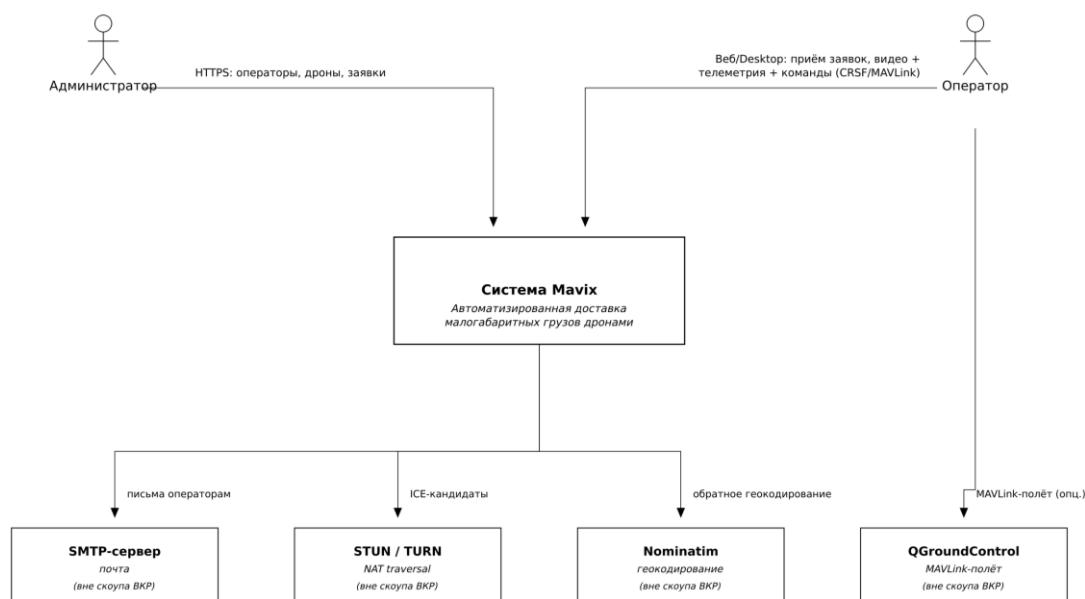


Рисунок 2.1 – Контекстная диаграмма системы (уровень C1)

2.2.2. Диаграмма контейнеров (C2)

Система декомпозируется на отдельно развёртываемые контейнеры: серверное приложение MavixServer (REST API и WebSocket-сигналинг), веб-кабинет MavixWeb, база данных PostgreSQL [27], а также клиенты – бортовой модуль и станция оператора (внешние по отношению к данной подтеме). Взаимодействие контейнеров и используемые протоколы показаны на рисунке 2.2.

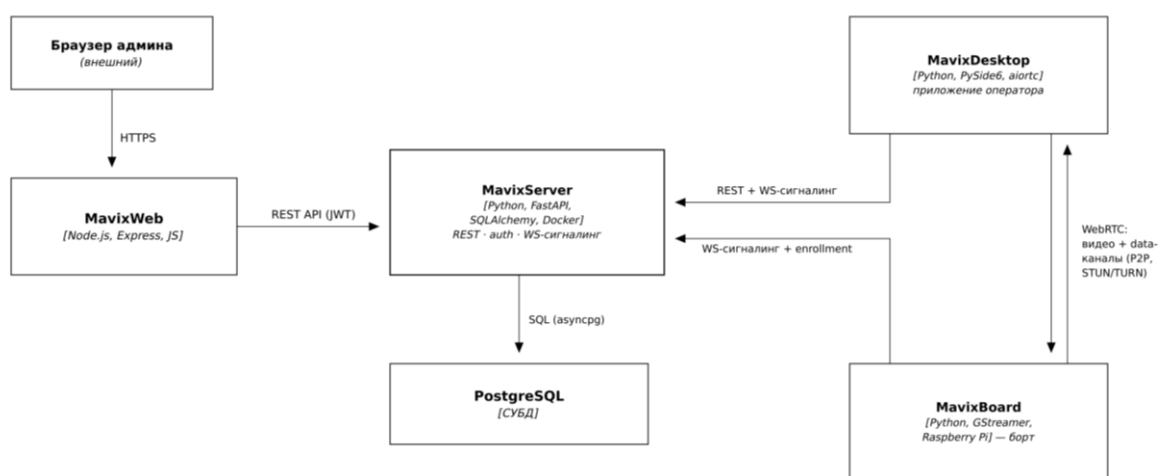


Рисунок 2.2 – Диаграмма контейнеров (уровень C2)

2.2.3. Компоненты сервера MavixServer (C3)

Контейнер MavixServer декомпозируется на компоненты: API-роутеры (обработка HTTP-запросов), сервисы (бизнес-логика), репозитории (доступ к данным), модели (SQLAlchemy [26]), а также отдельный слой WebSocket-сигналинга (обработчики, реестр соединений, ретранслятор, нотификатор). Диаграмма компонентов сервера приведена на рисунке 2.3.

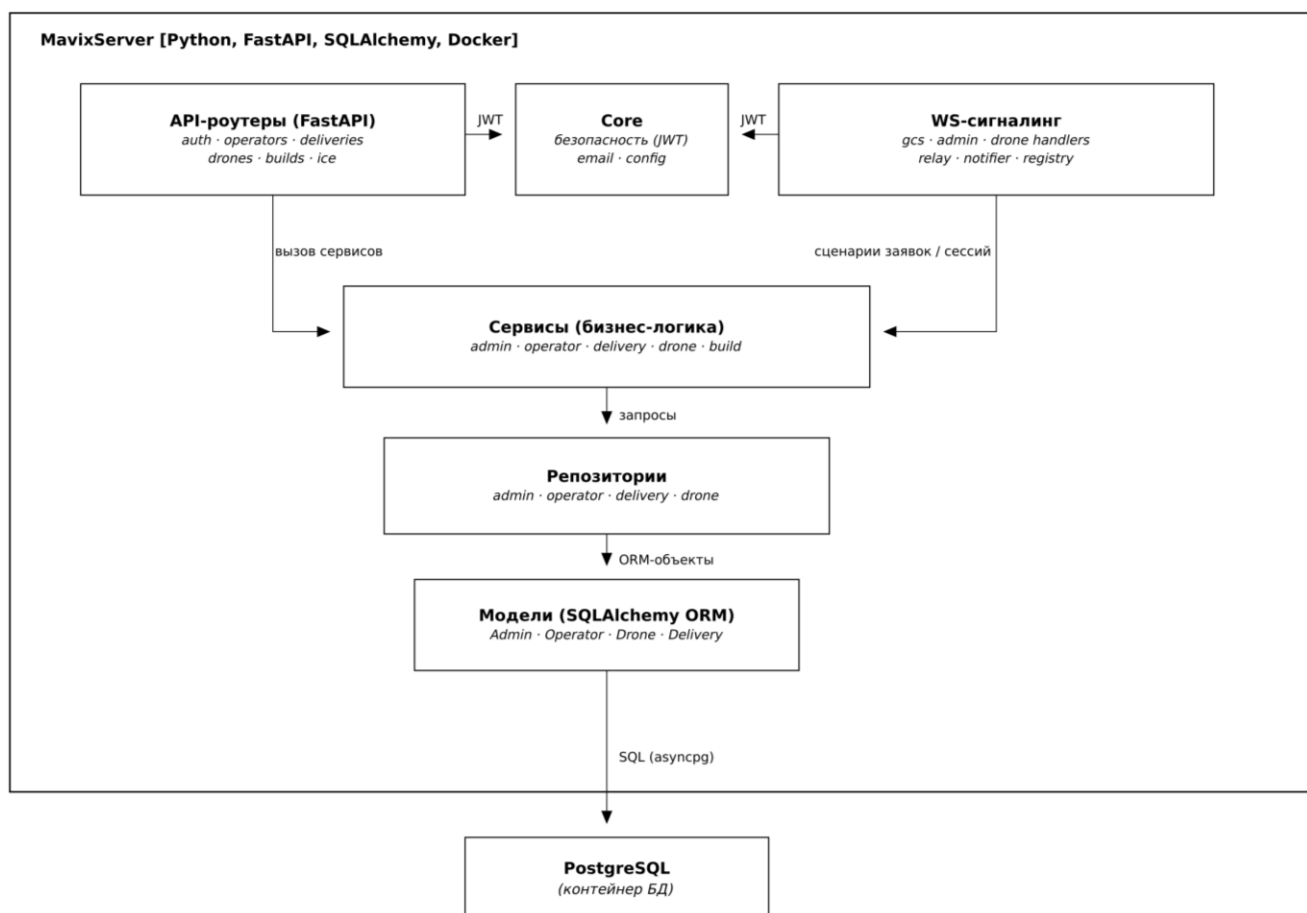


Рисунок 2.3 – Диаграмма компонентов сервера (уровень С3)

Взаимодействие компонентов организовано по принципу однонаправленной зависимости от верхних слоёв к нижним. API-роутеры принимают запрос, валидируют входные данные и вызывают соответствующий сервис; сервис реализует бизнес-правило и при необходимости обращается к одному или нескольким репозиториям; репозитории выполняют запросы к базе данных через средства отображения и возвращают доменные объекты. Обратные зависимости (от репозитория к сервисам или от сервисов к роутерам) отсутствуют, что обеспечивает низкую связанность и облегчает замену реализаций. Слой WebSocket-сигналинга работает параллельно REST-части и использует общий реестр соединений, через который обработчики маршрутизируют сообщения между оператором и бортом.

Декомпозиция на компоненты соответствует традиционной трёхслойной организации (представление – логика – данные) и допускает дальнейшую детализацию до уровня кода (уровень C4), рассматриваемого в подразделе 2.5. Такое соответствие между архитектурным описанием и структурой кода упрощает сопровождение и ввод в курс дела новых разработчиков.

2.2.4. Компоненты веб-кабинета MavixWeb (C3)

Второй контейнер информационной системы – веб-кабинет администратора MavixWeb. Он также декомпозируется до уровня компонентов, что позволяет рассматривать архитектуру каждого репозитория в отдельности, а не ограничиваться серверной частью. Контейнер делится на серверную и клиентскую части. Серверная часть – приложение на Node.js с фреймворком Express [30] (компонент `server.js`): оно отдаёт статические файлы, сопоставляет «чистые» адреса страниц с HTML-документами, внедряет в страницу конфигурацию через эндпоинт `/config.js` и проксирует обращения `/api` и `/ws` к серверу MavixServer, скрывая его адрес от браузера. Клиентская часть выполняется в браузере и реализована на ванильном JavaScript в виде набора модулей. Диаграмма компонентов веб-кабинета приведена на рисунке 2.4.

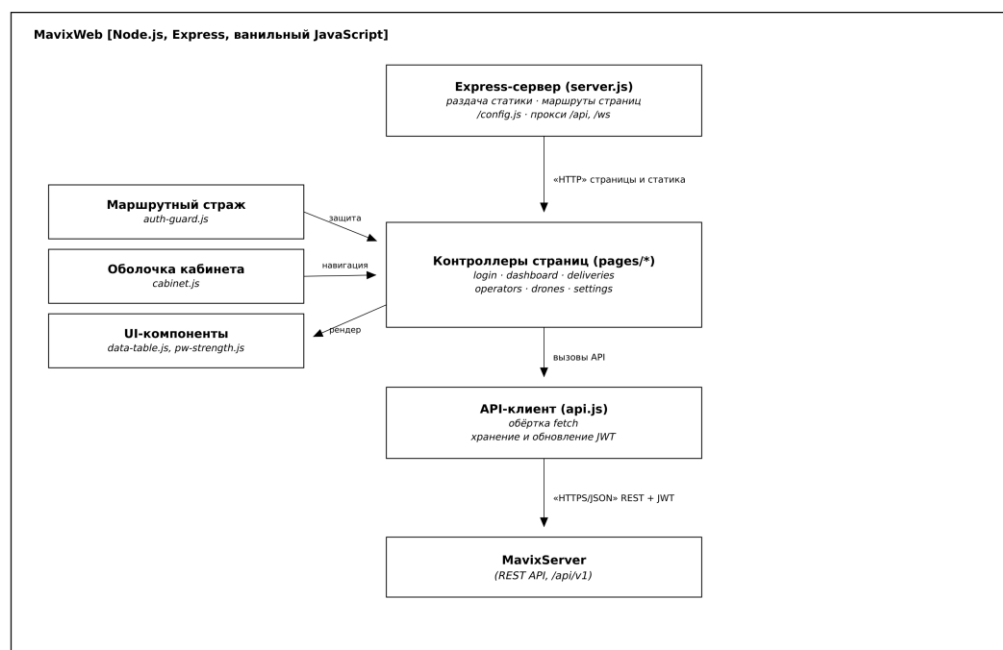


Рисунок 2.4 – Диаграмма компонентов веб-кабинета (уровень С3)

Ключевые клиентские компоненты: API-клиент (`api.js`) – единая обёртка над браузерным `fetch`, которая добавляет к запросам токен доступа, хранит пару токенов в локальном хранилище браузера и автоматически обновляет их по истечении срока; маршрутный страж (`auth-guard.js`), перенаправляющий неавторизованного пользователя на страницу входа; оболочка кабинета (`cabinet.js`), формирующая общую навигацию и шапку; контроллеры страниц (каталог `pages`) – по одному модулю на экран (вход, панель, доставки, операторы, дроны, настройки); а также переиспользуемые компоненты представления – конфигурируемая таблица `data-table.js` и индикатор надёжности пароля `pw-strength.js`. Контроллеры страниц обращаются к серверу только через API-клиент, что отделяет логику экранов от деталей сетевого взаимодействия.

Такая декомпозиция повторяет принцип разделения ответственности, принятый на сервере: транспорт (API-клиент), управление доступом (маршрутный страж), представление (контроллеры страниц и компоненты). Отказ от тяжёлого фронтенд-фреймворка в пользу модульного ванильного JavaScript оправдан масштабом задачи: кабинет администратора содержит ограниченный набор однотипных экранов, и введение сборщика и фреймворка усложнило бы проект без практической выгоды, что согласуется с принципами KISS и YAGNI.

2.3. Проектирование базы данных

2.3.1. Концептуальная и логическая модели данных

Концептуальная модель данных (рисунок 1.3) детализируется до логической: для каждой сущности определены атрибуты, первичные и внешние ключи. Сущность «администратор» (`admins`) владеет операторами, дронами и заявками. Сущность «оператор» (`operators`) хранит учётные данные и профиль. Сущность «дрон» (`drones`) описывает аппарат и его идентичность. Сущность «заявка» (`deliveries`) связывает дрон и оператора и содержит статус и снимки данных.

Нормализация выполнялась последовательно. Приведение к первой нормальной форме (1НФ) обеспечено атомарностью всех атрибутов: каждое поле хранит единственное значение, отсутствуют повторяющиеся группы и многозначные атрибуты. Приведение ко второй нормальной форме (2НФ) достигается тем, что все неключевые атрибуты функционально зависят от первичного ключа целиком; поскольку первичные ключи всех таблиц простые (суррогатные идентификаторы), частичные зависимости отсутствуют по построению. Приведение к третьей нормальной форме (3НФ) обеспечено устранением транзитивных зависимостей: неключевые атрибуты зависят только от ключа и не зависят друг от друга.

Сознательным и обоснованным отступлением от полной нормализации являются денормализованные снимки (`drone_name`, `operator_name`, `operator_passport`) в таблице заявок. Эти атрибуты дублируют данные таблиц дронов и операторов, что формально нарушает 3НФ, однако они необходимы для сохранности журнала доставок: при удалении дрона или оператора соответствующие внешние ключи заявки обнуляются (действие `ON DELETE SET NULL`), но историческая запись сохраняет имена участников на момент доставки. Без снимков журнал утратил бы значимую часть информации. Таким образом, денормализация здесь служит требованию сохранности данных и является осознанным проектным решением, а не упущением.

2.3.2. Физическая модель данных и выбор СУБД

В качестве системы управления базами данных выбрана реляционная СУБД PostgreSQL. Обоснование: предметная область имеет чёткую структуру и связи, критична целостность данных и поддержка транзакций (атомарный приём заявки), что является сильной стороной реляционных СУБД. PostgreSQL – зрелая СУБД с открытым кодом, поддержкой ограничений целостности, индексов и расширенных типов данных. Нереляционные СУБД отклонены, поскольку не дают преимуществ при структурированных данных и затрудняют поддержку транзакционной целостности.

Физическая модель учитывает типы PostgreSQL, индексы по первичным и внешним ключам, ограничения уникальности (email администратора, логин оператора) и перечислимый тип статуса заявки. Управление схемой выполняется миграциями Alembic [28]. Итоговая ER-диаграмма физической модели приведена ранее на рисунке 1.3. Состав основных таблиц базы данных и их ключевые поля приведены в таблице 2.2.

Таблица 2.2 – Состав основных таблиц базы данных

Таблица	Ключевые поля	Назначение
admins	admin_id (PK), email (UK), password, enrollment_token	учётные записи администраторов
operators	operator_id (PK), admin_id (FK), username (UK), password, full_name, passport, is_active	учётные записи операторов
drones	drone_id (PK), admin_id (FK), name, drone_token, last_seen_at	зарегистрированные борта
deliveries	delivery_id (PK), admin_id (FK), drone_id (FK, SET NULL), operator_id (FK, SET NULL), status, drone_name, operator_name, address, created_at	заявки на доставку и журнал

Описанная структура таблиц реализует логическую модель данных средствами выбранной СУБД и поддерживает целостность данных за счёт первичных и внешних ключей с заданными правилами каскадного изменения.

2.4. Проектирование пользовательского интерфейса

Пользовательский интерфейс информационной системы – веб-кабинет администратора (MavixWeb). Спроектированы ключевые экранные формы: страница входа и регистрации, раздел «Операторы» (список, создание, удаление), раздел «Дроны» (список, удаление), раздел «Доставки» (создание заявки с выбором точки на карте, журнал со статусами), страница скачивания дистрибутивов клиентов. Навигация организована через боковое меню; создание заявки сопровождается выбором точки доставки на интерактивной карте с

автоматическим определением адреса. Прототипы экранных форм и их связь с вариантами использования приведены в приложении.

Страница входа содержит форму с полями адреса электронной почты и пароля и переключением между входом и регистрацией. Главный экран кабинета разделён на боковое меню (разделы «Доставки», «Операторы», «Дроны», «Скачать») и рабочую область. Раздел «Операторы» представлен таблицей с действиями создания и удаления; при создании оператора форма запрашивает фамилию, имя, паспортные данные и адрес, а сгенерированные логин и пароль показываются администратору и направляются оператору. Раздел «Доставки» содержит форму создания заявки (выбор дрона из выпадающего списка, поле адреса и карта для выбора точки) и журнал доставок с цветовой индикацией статусов. Эргономика интерфейса соответствует принципам юзабилити: единообразие элементов, обратная связь по действиям, защита от случайных удалений через подтверждение.

2.5. Детальное проектирование (уровень кода)

2.5.1. Диаграммы классов ключевых компонентов

Доменная модель сервера представлена набором классов-сущностей SQLAlchemy. Диаграмма классов доменной модели [7] приведена на рисунке 2.5, а детальная модель центральной сущности «заявка» вместе с перечислением статусов – на рисунке 2.6.

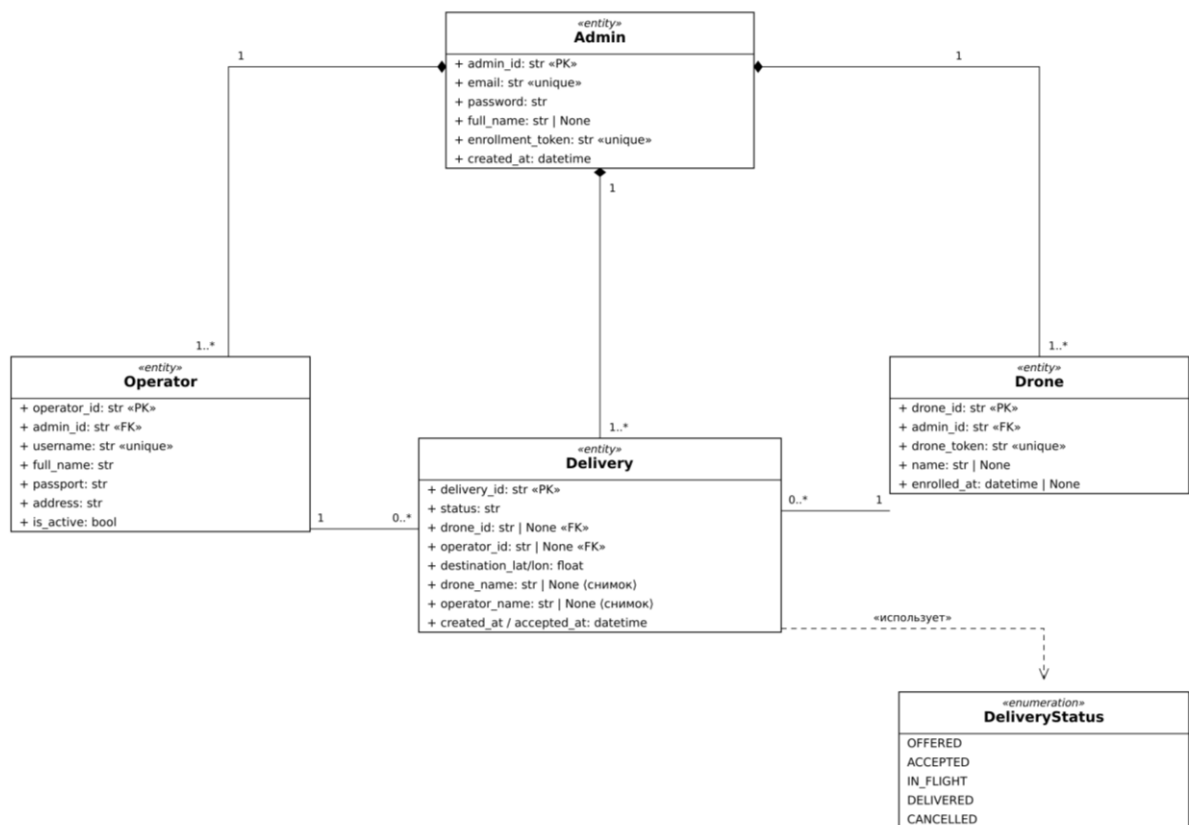


Рисунок 2.5 – Диаграмма классов доменной модели

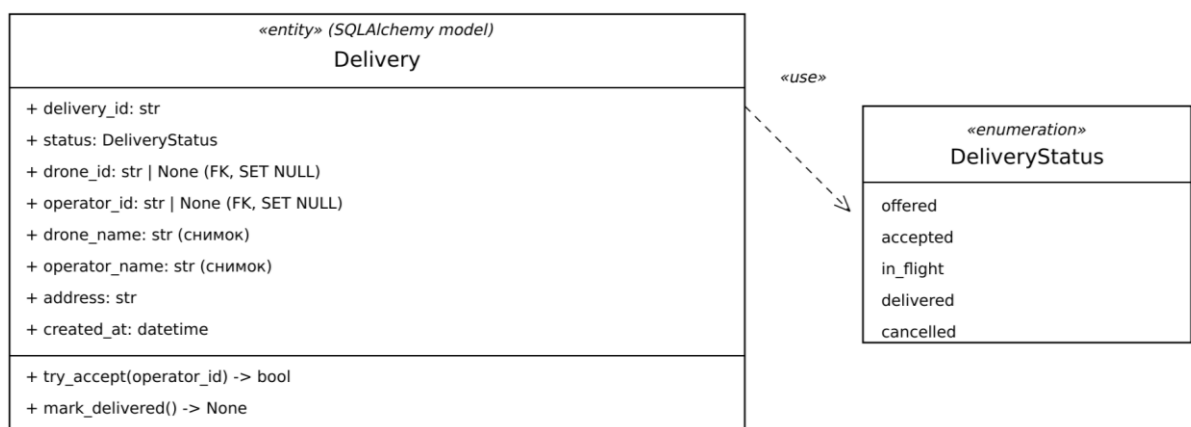


Рисунок 2.6 – Класс Delivery и перечисление статусов доставки

2.5.2. Диаграммы последовательности для основных сценариев

Наиболее ответственным сценарием является приём заявки оператором: при одновременной попытке нескольких операторов заявка должна быть

закреплена ровно за одним. Диаграмма последовательности этого сценария приведена на рисунке 2.7.

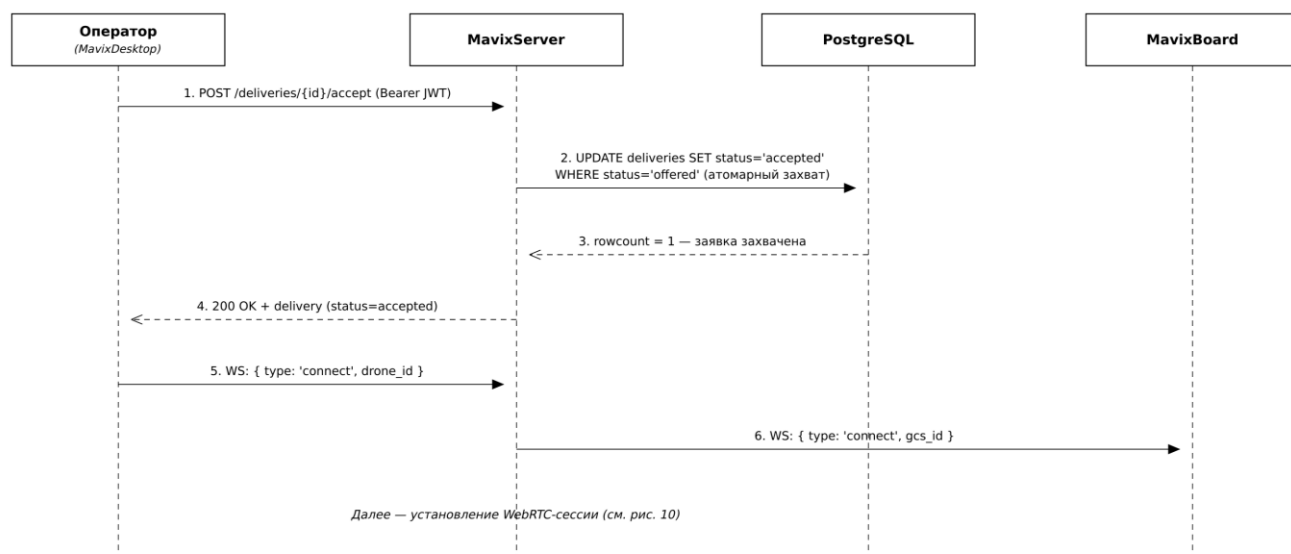


Рисунок 2.7 – Диаграмма последовательности приёма заявки

Вторым ответственным сценарием со стороны информационной системы является посредничество при установлении соединения между оператором и бортом. Сервер не передаёт медиапоток, а маршрутизирует через слой сигналинга служебные сообщения (предложение, ответ, ICE-кандидаты), после чего стороны поднимают прямой канал P2P. Тем самым нагрузка на сервер ограничивается короткой фазой установления связи, а сам обмен медиаданными в зону ответственности информационной системы не входит.

2.5.3. Применение шаблонов проектирования

При проектировании информационной системы применён ряд шаблонов проектирования — как классических порождающих, структурных и поведенческих шаблонов «банды четырёх» (GoF) [6], так и архитектурных шаблонов корпоративных приложений. Их применение обеспечивает повторное использование проверенных решений и повышает сопровождаемость кода.

Шаблон Repository (репозиторий) [4] изолирует доступ к данным в отдельном слое, скрывая детали формирования SQL-запросов и средств

отображения от бизнес-логики. Каждой сущности соответствует свой репозиторий, предоставляющий операции выборки, создания, обновления и удаления. Это позволяет изменять способ хранения, не затрагивая сервисы, и подменять репозитории заглушками при тестировании.

Шаблон Service Layer (слой служб) выделяет бизнес-правила в отдельный слой сервисов, расположенный между роутерами и репозиториями. Сервисы координируют работу нескольких репозиториях в рамках одной операции (например, при приёме заявки обновляют её статусы и формируют снимки), инкапсулируя логику предметной области. Шаблон Dependency Injection (внедрение зависимостей) реализован средствами фреймворка: сессия базы данных и текущий пользователь передаются в обработчики и сервисы извне, что снижает связанность и упрощает тестирование.

Дополнительно применён приём атомарного «захвата» для разрешения состязания при приёме заявки несколькими операторами (поведенческое решение на уровне базы данных), а также шаблон Data Mapper, реализуемый средствами объектно-реляционного отображения SQLAlchemy, который сопоставляет доменные объекты строкам таблиц. Перечисленные решения детализируются и иллюстрируются фрагментами кода в разделе 3 и приложении Г.

2.6. Обоснование выбора стека технологий

Язык серверной части – Python: он быстро связывает разнородные библиотеки, имеет развитую экосистему для веб-разработки и асинхронного программирования. Веб-фреймворк – FastAPI [25]: асинхронный, с автоматической валидацией данных через Pydantic [29] и генерацией документации OpenAPI [31]. Доступ к данным – SQLAlchemy 2.0 в асинхронном режиме, миграции – Alembic. СУБД – PostgreSQL. Сигналинг – протокол WebSocket поверх того же сервера.

Веб-кабинет реализован на Node.js с фреймворком Express (раздача статики и проксирование к API) и клиентском ванильном JavaScript без тяжёлого

фреймворка, что упрощает сопровождение и снижает размер сборки. Контейнеризация – Docker [32] и Docker Compose; обратный прокси с TLS – Caddy. Инструменты разработки: система контроля версий Git [16], тестовый фреймворк pytest [33], средство нагрузочного тестирования Locust [34]. Сводка технологического стека приведена в таблице 2.3.

Таблица 2.3 – Технологический стек информационной системы

Слой	Технология	Назначение
Веб-API	Python, FastAPI	REST-эндпоинты, валидация, OpenAPI
Доступ к данным	SQLAlchemy 2.0, Alembic	ORM, миграции схемы
СУБД	PostgreSQL 16	хранение данных, транзакции
Сигналинг	WebSocket	обмен offer/answer/ICE, оповещения
Веб-кабинет	Node.js, Express, JavaScript	интерфейс администратора
Инфраструктура	Docker, Caddy	контейнеризация, TLS-прокси
Качество	pytest, Locust	тестирование и нагрузка

Выбранный стек опирается на зрелые открытые технологии, что снижает стоимость владения, упрощает подбор специалистов и обеспечивает переносимость системы между окружениями.

Выводы по главе 2

В результате проектирования обоснован архитектурный стиль системы – модульный монолит с многоуровневой серверной частью, дополненный P2P-передачей медиа по WebRTC и событийным сигналингом по WebSocket. Архитектура описана по модели C4 [13] на трёх уровнях (контекст, контейнеры, компоненты). Спроектирована база данных в третьей нормальной форме с обоснованной денормализацией снимков, выбрана СУБД PostgreSQL. Разработаны диаграммы классов и последовательности, определены применяемые шаблоны проектирования (Repository, Service Layer, Dependency Injection), обоснован стек технологий. Архитектура готова к этапу реализации.

ГЛАВА 3. КОНСТРУИРОВАНИЕ (РЕАЛИЗАЦИЯ) ПРОГРАММНОГО ПРОДУКТА

3.1. Организация процесса разработки

3.1.1. Методология разработки

Разработка велась по гибкой методологии [1] в варианте Kanban, адаптированном к условиям индивидуальной работы: задачи фиксировались в виде упорядоченного списка, выполнялись небольшими инкрементами с регулярной интеграцией результата. Такой подход обеспечил постепенное наращивание функциональности и раннее выявление дефектов.

3.1.2. Система контроля версий и стратегия ветвления

В качестве распределённой системы контроля версий выбран Git; репозиторий размещён на платформе GitHub (<https://github.com/AurunChill/Mavix>). Применена модель ветвления, близкая к GitHub Flow [17]: стабильная ветка master, ветка разработки и тематические ветки (delivery_control и др.) для крупных направлений работ. Слияние выполнялось через запросы на слияние с просмотром изменений. Сообщения коммитов оформлялись по соглашению Conventional Commits [18] (feat, fix, docs, test, chore), что упрощает анализ истории.

Репозиторий проекта организован как набор связанных компонентов; серверная часть и веб-кабинет ведутся в отдельных каталогах с собственной историей. История коммитов отражает поэтапное наращивание функциональности: создание базовой структуры, реализация моделей и репозиториев, добавление сервисов и REST-эндпоинтов, реализация сигналинга, написание тестов и документации. Осмысленные сообщения коммитов позволяют проследить ход работ и при необходимости откатиться к стабильному состоянию. Регулярность коммитов и наличие тематических веток подтверждают применение инженерного подхода к управлению исходным кодом.

3.2. Соответствие реализации спроектированной архитектуре

3.2.1. Структура проекта

Файловая структура серверного приложения отражает многоуровневую архитектуру и компоненты диаграммы СЗ. Каталог `api` содержит REST-роутеры, `services` – бизнес-логику, `repositories` – доступ к данным, `models` – сущности SQLAlchemy, `ws` – слой WebSocket-сигналинга, `core` – сквозные модули (конфигурация, безопасность, база данных, почта). Веб-кабинет организован по страницам и переиспользуемым компонентам.

Структура каталогов серверного приложения:

```
MavixServer/src/mavixserver/  
  api/           # REST-роутеры (контроллеры)  
  services/      # бизнес-логика  
  repositories/  # доступ к данным  
  models/        # сущности SQLAlchemy  
  ws/            # WebSocket-сигналинг  
  core/          # конфигурация, безопасность, БД, почта  
  __main__.py    # сборка приложения FastAPI
```

3.2.2. Конфигурация приложения

Управление конфигурацией реализовано через класс настроек на основе Pydantic Settings: параметры читаются из переменных окружения и файла `.env` (адрес базы данных, секрет JWT, параметры STUN/TURN, SMTP). Это разделяет код и конфигурацию и обеспечивает разные настройки для окружений разработки и эксплуатации.

3.3. Детализация ключевых компонентов (уровень С4 – Code)

3.3.1. Компонент управления заявками

Компонент управления заявками реализует центральный сценарий системы – жизненный цикл заявки на доставку. Назначение: создание заявки администратором, рассылка операторам, атомарный приём и смена статусов. Ключевой класс – сущность `Delivery` (рисунок 2.6); бизнес-логика вынесена в сервис `delivery`, доступ к данным – в репозиторий `delivery`.

Наиболее ответственный фрагмент – атомарный приём заявки. Чтобы при одновременной попытке нескольких операторов заявка была закреплена ровно

за одним, приём реализован одним UPDATE с проверкой условия и числа изменённых строк, без явных блокировок (листинг 3.1).

Листинг 3.1 – Атомарный приём заявки

```
async def try_accept(session, delivery_id, operator):
    # атомарно: побеждает первый успевший запрос
    result = await session.execute(
        update(Delivery)
        .where(Delivery.delivery_id == delivery_id,
              Delivery.status == 'offered')
        .values(status='accepted',
              operator_id=operator.operator_id,
              operator_name=operator.full_name))
    await session.commit()
    return result.rowcount == 1    # True -> заявка захвачена
```

Алгоритм приёма заявки в виде блок-схемы приведён на рисунке 3.1.

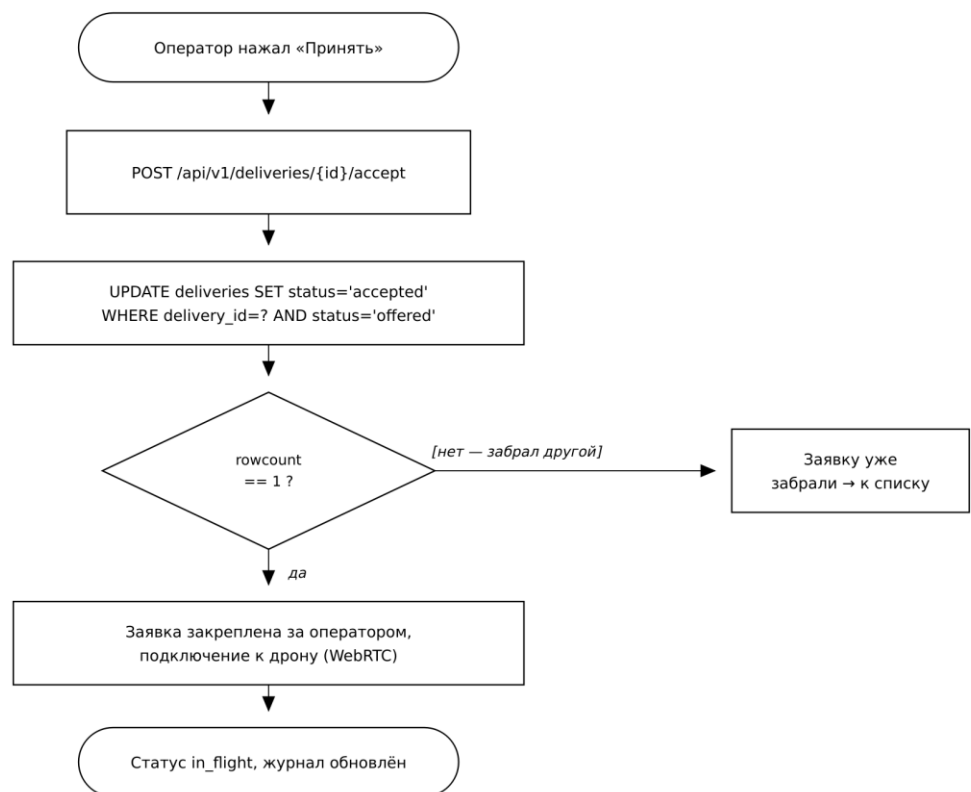


Рисунок 3.1 – Алгоритм атомарного приёма заявки

3.3.2. Компонент сигналинга WebRTC

Компонент сигналинга обеспечивает установление соединения между оператором и бортом. По протоколу WebSocket стороны обмениваются предложением (offer), ответом (answer) и ICE-кандидатами; сервер

маршрутизирует сообщения через реестр соединений и не участвует в передаче медиапотока. Это реализует событийный (event-driven) стиль взаимодействия.

3.3.3. Компонент доступа к данным

Слой доступа к данным реализует шаблон Repository: каждый репозиторий инкапсулирует запросы к одной сущности и возвращает доменные объекты. Объектно-реляционное отображение выполняется средствами SQLAlchemy 2.0. Выборка по идентификатору реализована методом `get_by_id`, возвращающим доменный объект или значение `None`.

3.3.4. Компонент управления учётными записями

Компонент управления учётными записями реализует регистрацию и вход администратора, а также создание и блокировку операторов. При создании оператора логин и пароль генерируются автоматически и направляются на указанный адрес электронной почты; пароль хранится в виде хеша. Вход возвращает пару токенов (`access` и `refresh`). Для борта предусмотрена самостоятельная регистрация (`enrollment`) по выданному администратору токеном. Логика вынесена в сервисы `admin` и `operator`, что соответствует шаблону Service Layer.

3.3.5. Реализация слоя сигналинга

Слой сигналинга реализован поверх WebSocket-маршрута. При подключении клиент проходит аутентификацию по токеном и регистрируется в реестре соединений; входящие сообщения (`offer`, `answer`, ICE-кандидаты) маршрутизируются адресату через ретранслятор, а оповещения о новых заявках рассылаются операторам нотификатором. Такая организация реализует событийный стиль взаимодействия и отделяет логику сигналинга от REST-части.

3.3.6. Учёт ресурсов и ведение журнала

Компонент учёта реализует ведение справочников операторов и дронов и журнала доставок. Создание оператора сопровождается генерацией учётных данных и отправкой их по электронной почте; блокировка и удаление учётной

записи выполняются с проверкой принадлежности администратору. Дрон регистрируется самостоятельно при первом запуске и далее отображается в справочнике; для каждого аппарата фиксируется время последней активности. Журнал доставок формируется автоматически по мере смены статусов заявок и сохраняет историческую информацию благодаря денормализованным снимкам.

Отдельной задачей сопровождения является поддержание актуальности справочника дронов. Для этого предусмотрена фоновая процедура очистки, периодически удаляющая аппараты, не выходившие на связь дольше заданного срока, и сохраняющая при этом онлайн-устройства. Процедура запускается по расписанию в рамках жизненного цикла приложения и не требует вмешательства администратора, что повышает надёжность учёта и предотвращает накопление неактуальных записей.

3.4. Реализация клиентской части (веб-кабинет администратора)

Помимо серверного приложения, в составе информационной системы реализован веб-кабинет администратора MavixWeb. Он представляет собой самостоятельный компонент со своей структурой и историей разработки, поэтому его реализация рассматривается отдельно от серверной части.

3.4.1. Серверное приложение и маршрутизация

Серверная часть веб-кабинета построена на платформе Node.js с фреймворком Express и выполняет три задачи. Во-первых, она отдаёт статические ресурсы (HTML-страницы, таблицы стилей, клиентские сценарии). Во-вторых, сопоставляет «чистые» адреса вида /deliveries с соответствующими HTML-документами, что обеспечивает удобную навигацию без расширений файлов. В-третьих, она проксирует все обращения к путям /api и /ws на серверное приложение MavixServer. Проксирование выбрано осознанно: браузер обращается только к своему источнику, благодаря чему адрес внутреннего API не раскрывается, а ограничения политики единого источника (CORS) не возникают. Адрес целевого API не «зашифрован» в код, а передаётся в страницу через

служебный эндпоинт /config.js, что позволяет менять окружение без пересборки клиента.

3.4.2. Организация клиентского кода

Клиентская часть выполняется в браузере и написана на ванильном JavaScript без применения тяжёлого фреймворка и без шага сборки. Код разделён на самостоятельные модули, каждый из которых инкапсулирует своё состояние с помощью немедленно вызываемой функции. Базовый модуль api.js реализует единую обёртку над браузерным интерфейсом fetch: он добавляет к защищённым запросам токен доступа, хранит пару токенов (доступа и обновления) в локальном хранилище браузера, отслеживает срок их действия и заблаговременно обновляет токен, освобождая остальной код от деталей авторизации. Модуль auth-guard.js играет роль маршрутного стража и перенаправляет неавторизованного пользователя на страницу входа; модуль cabinet.js формирует общую оболочку кабинета – боковое меню и шапку. Логика отдельных экранов вынесена в каталог pages: по одному контроллеру на страницу (вход, панель, доставки, операторы, дроны, настройки).

Для устранения дублирования повторяющиеся элементы интерфейса оформлены как переиспользуемые компоненты. Так, модуль data-table.js реализует конфигурируемую таблицу: вызывающий код передаёт описание столбцов и данные, а компонент берёт на себя отрисовку, поиск и постраничный вывод. За счёт этого таблицы операторов, дронов и доставок строятся на единой основе, отличаясь только конфигурацией, что соответствует принципу DRY. Отдельный модуль pw-strength.js оценивает надёжность пароля при регистрации. Контроллеры страниц обращаются к серверу исключительно через модуль api.js, что отделяет логику экранов от деталей сетевого взаимодействия и упрощает изменение того и другого независимо.

3.4.3. Реализация экранных форм

Центральный экран кабинета – форма создания заявки на доставку. Администратор выбирает дрон из выпадающего списка, указывает адрес

назначения и, при необходимости, адрес отправления. Поле адреса снабжено подсказками: по мере ввода клиент обращается к геокодеру OpenStreetMap (Nominatim) и предлагает варианты, а выбранную точку дублирует на интерактивной карте, построенной средствами библиотеки Leaflet. Это снижает вероятность ошибки и ускоряет ввод. Разметка поля адреса и его базовый стиль приведены в листингах 3.2 и 3.3; класс `input` используется единообразно для всех полей ввода и выпадающих списков формы.

Листинг 3.2 – Разметка поля ввода адреса (deliveries.html)

```
<label class="form-label"
      for="dl-dest-address">Адрес назначения</label>
<div class="addr-field" data-role="addr-field">
  <input class="input" id="dl-dest-address"
        name="destination_address" type="text"
        autocomplete="off"
        placeholder="Начните вводить адрес...">
  <div class="addr-suggest"
        data-role="addr-suggest" hidden></div>
</div>
```

Листинг 3.3 – Стиль поля ввода (css/components.css)

```
.input, .textarea {
  width: 100%;
  padding: 11px 14px;
  background: var(--bg-elev-1);
  border: 1px solid var(--border-strong);
  border-radius: var(--r-md);
  font-size: var(--fs-base);
}
.input:focus {
  outline: none;
  border-color: var(--accent);
  box-shadow: 0 0 0 3px var(--accent-soft);
}
```

Стилизация построена на CSS-переменных (цвета, отступы, скругления), что обеспечивает единое оформление всех экранов и позволяет менять тему в одном месте. Журнал доставок и справочники операторов и дронов отрисовываются компонентом `data-table.js` и снабжены поиском и постраничным выводом. Экранные формы веб-кабинета приведены в приложении Е.

3.5. Демонстрация применения принципов качественного кода

3.5.1. Принципы SOLID

При разработке серверной части последовательно применялись пять принципов SOLID [2], формирующих основу поддерживаемого объектно-ориентированного кода. Ниже каждый принцип рассмотрен на конкретных примерах из реализации.

Принцип единственной ответственности (Single Responsibility, S) реализован через строгое разделение на слои. Каждый модуль решает ровно одну задачу: репозитории (каталог repositories) отвечают исключительно за доступ к данным и формирование SQL-запросов, сервисы (каталог services) – за бизнес-правила, такие как проверка условий смены статуса и формирование снимков, а роутеры (каталог api) – за приём HTTP-запроса, валидацию и формирование ответа. Благодаря этому изменение способа хранения данных затрагивает только репозитории, изменение бизнес-правил – только сервисы, а изменение формата обмена – только роутеры, что снижает связанность и риск регрессий.

Принцип открытости/закрытости (Open/Closed, O) проявляется в том, что код открыт для расширения, но закрыт для модификации. Жизненный цикл заявки задан перечислением статусов и единой точкой их смены; добавление нового статуса или правила перехода выполняется расширением перечисления и таблицы переходов без правки существующих обработчиков. Аналогично, добавление новой сущности предметной области сводится к созданию отдельного модуля репозитория и сервиса, не затрагивая уже реализованные компоненты.

Принцип подстановки Лисков (Liskov Substitution, L) обеспечивается корректной иерархией наследования: все модели-сущности наследуют общий декларативный базовый класс и единообразно используются репозиториями и средствами отображения. В тестовой среде объект сессии базы данных и репозитории заменяются совместимыми заглушками, и сервисы продолжают работать без изменений, что подтверждает соблюдение принципа.

Принцип разделения интерфейсов (Interface Segregation, I) выражается в том, что вместо «толстых» универсальных модулей сервисы представлены набором узких функций-операций (зарегистрировать, войти, создать, принять, сменить статус), а обработчики запрашивают через зависимости только те ресурсы, которые им действительно нужны (сессию, текущего пользователя). Клиенты не зависят от методов, которые не используют.

Принцип инверсии зависимостей (Dependency Inversion, D) реализован механизмом внедрения зависимостей фреймворка: сервисы и обработчики не создают соединения с базой данных самостоятельно, а получают сессию извне через механизм Depends. Это инвертирует направление зависимости – высокоуровневая бизнес-логика не зависит от деталей создания инфраструктурных объектов, что упрощает замену реализаций и тестирование.

3.5.2. Применение шаблонов проектирования

Использованы шаблоны: Repository и Service Layer (разделение доступа к данным и бизнес-логики), Dependency Injection (внедрение сессии и текущего пользователя), а также приём атомарного «такси»-захвата для разрешения гонки при приёме заявки. В клиентской части веб-кабинета применён приём переиспользуемого компонента таблицы данных, настраиваемого данными.

3.5.3. Принципы чистого кода

Код следует принципам чистого кода [3, 5]: осмысленные имена сущностей и функций, небольшие функции с единственной задачей, самодокументируемость и комментарии только там, где они поясняют неочевидные решения. Человеческочитаемые сообщения (ошибки, логи) ведутся на русском языке. Стилль кода контролируется линтером и статической проверкой типов.

3.6. Интеграция с внешними сервисами и библиотеками

Система интегрируется с внешним почтовым сервером по протоколу SMTP (с шифрованием соединения) для отправки уведомлений: приветственное

письмо администратору, выдача учётных данных оператору, уведомление о регистрации дрона. Для обхода NAT при установлении WebRTC-соединения используются внешние серверы STUN и TURN; их адреса выдаются клиентам по запросу через эндпоинт ice-servers. Обработка ошибок внешних вызовов предусматривает тайм-ауты и устойчивость к недоступности почтового сервиса (сбой отправки письма не прерывает основную операцию).

Из сторонних библиотек ключевыми являются: FastAPI (веб-фреймворк), SQLAlchemy и asyncpg (доступ к PostgreSQL), Alembic (миграции), passlib (хеширование паролей), PyJWT (токены), aiosmtplib (асинхронная отправка почты), pydantic (валидация данных). Выбор обусловлен зрелостью, поддержкой асинхронности и распространённостью в экосистеме Python. Перечень ключевых библиотек и их назначение приведены в таблице 3.1.

Таблица 3.1 – Ключевые сторонние библиотеки серверной части

Библиотека	Назначение
FastAPI	асинхронный веб-фреймворк, REST и валидация
SQLAlchemy, asyncpg	ORM и асинхронный драйвер PostgreSQL
Alembic	миграции схемы базы данных
passlib (bcrypt)	хеширование паролей
PyJWT	выпуск и проверка JWT-токенов
aiosmtplib	асинхронная отправка электронной почты
pydantic	валидация и сериализация данных
pytest, pytest-cov, Locust	тестирование и измерение покрытия

Все используемые библиотеки распространяются под свободными лицензиями и зафиксированы по версиям в файле зависимостей, что обеспечивает воспроизводимость сборки и контролируемое обновление.

3.7. Обеспечение безопасности и обработка ошибок

3.7.1. Аутентификация и авторизация

Аутентификация реализована на основе JWT-токенов: при входе выдаются access- и refresh-токены, в полезной нагрузке access-токена содержится идентификатор и роль (администратор или оператор). Защита маршрутов обеспечивается зависимостями, проверяющими подлинность токена и роль.

Самостоятельная регистрация борта выполняется по отдельному enrollment-токену, не являющемуся JWT и выдаваемому администратору при регистрации. Проверка подлинности и роли вынесена в зависимость, подключаемую к защищённым эндпоинтам (листинг 3.4).

Листинг 3.4 – Защита маршрута и проверка роли

```
async def current_admin(
    token: str = Depends(oauth2_scheme),
    session: AsyncSession = Depends(get_session)):
    payload = security.decode_token(token)
    if payload.get('role') != 'admin':
        raise HTTPException(status_code=403,
                             detail='Недостаточно прав')
    admin = await admin_repo.get_by_id(
        session, payload['sub'])
    if admin is None:
        raise HTTPException(status_code=401)
    return admin
```

3.7.2. Валидация входных данных

Входные данные валидируются на серверной стороне средствами Pydantic: для каждого эндпоинта определена схема запроса, некорректные данные отклоняются с кодом 422. Защита от SQL-инъекций обеспечивается параметризованными запросами и отказом от конкатенации строк при работе с СУБД (ORM SQLAlchemy). Для веб-кабинета предусмотрена проверка данных и на клиентской стороне.

3.7.3. Логирование и мониторинг

Логирование ведётся стандартным модулем logging с уровнями DEBUG, INFO, WARN, ERROR. Логируются вход и обновление токенов, создание и смена статуса заявок, регистрация бортов, события сигналинга, ошибки базы данных и внешних сервисов. Записи снабжены префиксами компонентов и выводятся в стандартный поток, собираемый средствами Docker.

3.7.4. Обработка исключительных ситуаций

Применена централизованная стратегия обработки ошибок: бизнес-ошибки возбуждаются как исключения HTTP с понятным сообщением и корректным кодом состояния (401, 403, 404, 409), не раскрывая внутреннюю

структуру системы. Непредвиденные ошибки логируются и преобразуются в ответ 500.

3.8. Версионность и управление конфигурацией

Версионирование продукта ведётся по схеме семантического версионирования [19] (MAJOR.MINOR.PATCH): мажорная версия повышается при несовместимых изменениях API, минорная – при добавлении функциональности, патч – при исправлениях. Зависимости фиксируются в файле проекта (package.json) для серверной части и в package.json – для веб-кабинета. Воспроизводимость окружения обеспечивается контейнеризацией: образы собираются по Dockerfile, состав сервисов (приложение, база данных, прокси) описан в docker-compose. Конфигурации окружений разделены через файлы переменных окружения.

Выводы по главе 3

В результате этапа конструирования реализована информационная система в соответствии со спроектированной архитектурой. Процесс разработки организован с использованием Git и осмысленных коммитов. Реализованы ключевые компоненты: управление заявками с атомарным приёмом, сигналинг WebRTC, слой доступа к данным. При разработке соблюдены принципы SOLID и чистого кода, применены шаблоны Repository, Service Layer и Dependency Injection. Внедрены механизмы аутентификации на основе JWT, валидации данных, логирования и централизованной обработки ошибок. Реализованная функциональность покрывает функциональные требования раздела 1; продукт готов к тестированию.

ГЛАВА 4. ТЕСТИРОВАНИЕ, ОБЕСПЕЧЕНИЕ КАЧЕСТВА И СОПРОВОЖДЕНИЕ ПРОГРАММНОГО ПРОДУКТА

4.1. Стратегия тестирования

Цель тестирования – подтвердить соответствие реализованной системы требованиям раздела 1 и оценить её качество. Критерий начала тестирования – готовность компонента; критерий окончания – прохождение всех тестов и достижение целевого покрытия кода не ниже 70%. Применены три уровня тестирования: модульное (отдельные функции, репозитории, сервисы), интеграционное (взаимодействие через REST API и с базой данных) и системное (сквозные сценарии). Из типов тестирования выполнены функциональное, нагрузочное и тестирование безопасности механизмов аутентификации.

Уровни тестирования выбраны исходя из архитектуры системы. Модульное тестирование проверяет отдельные функции, методы репозитория и сервисов в изоляции от внешних зависимостей и нацелено на раннее выявление логических ошибок. Интеграционное тестирование проверяет совместную работу слоёв – роутеров, сервисов и репозитория – а также корректность взаимодействия с базой данных (запросы, транзакции). Системное тестирование охватывает сквозные пользовательские сценарии (от создания заявки до фиксации доставки) и проверяет систему как единое целое.

Типы тестирования подобраны под характеристики веб-ориентированной информационной системы. Функциональное тестирование подтверждает реализацию функциональных требований в позитивных и негативных сценариях. Нагрузочное тестирование оценивает производительность сервера под потоком обращений. Тестирование безопасности проверяет механизмы аутентификации и авторизации, а также устойчивость к типовым атакам. Регрессионное тестирование выполняется повторным прогоном всего набора после изменений и предотвращает повторное появление устранённых дефектов.

Инструментарий обоснован зрелостью и распространённостью в экосистеме выбранного стека. Модульные и интеграционные тесты серверной

части написаны на фреймворке pytest; для проверки REST API применяется тестовый клиент TestClient, позволяющий обращаться к приложению без поднятия реального сетевого сервера; покрытие кода измеряется библиотекой pytest-cov; нагрузочное тестирование выполняется средством Locust; тесты веб-кабинета написаны на фреймворке Jest. Тестовая среда использует встроенную СУБД SQLite в оперативной памяти, что исключает зависимость от внешней базы данных и обеспечивает быстрый и воспроизводимый прогон; внешние сервисы (почтовый сервер) заменяются заглушками. Тестовые данные формируются фикстурами, создающими изолированное состояние базы для каждого теста.

4.2. Модульное тестирование

Модульные тесты организованы зеркально структуре проекта: каталоги тестов соответствуют слоям (модели, репозитории, сервисы, API). Соглашение по именованию – описательные имена, отражающие проверяемое условие и ожидаемый результат. Для изоляции компонентов от внешних зависимостей используются заглушки. Разработаны тесты бизнес-логики (сервисы администратора, оператора, дронов, заявок), доступа к данным (репозитории) и валидации.

Измерение покрытия серверной части выполнено средством pytest-cov, веб-кабинета – средством Jest с флагом сбора покрытия. Достигнутые показатели по компонентам информационной системы сведены в таблицу 4.1 и показаны на рисунке 4.1.

Таблица 4.1 – Результаты модульного и интеграционного тестирования

Компонент	Количество тестов	Покрытие
MavixServer (сервер)	341	80%
MavixWeb (веб-кабинет)	40	85%

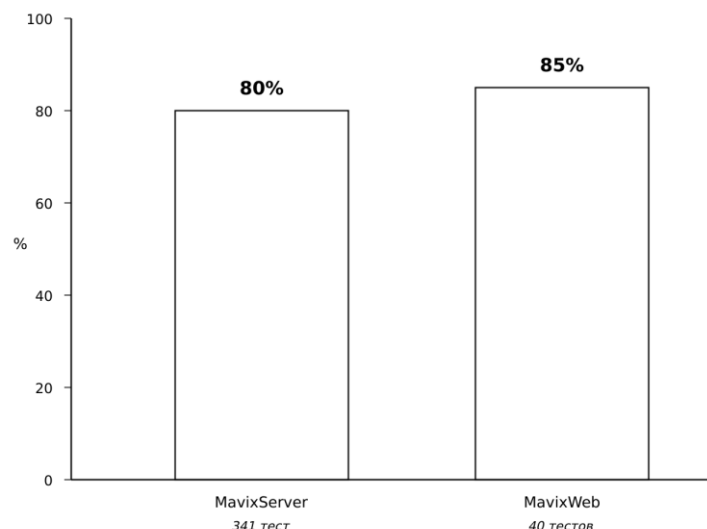


Рисунок 4.1 – Покрытие кода тестами (сервер и веб-кабинет)

Примечание: три тестовых файла слоя сигналинга, поднимающие WebSocket-клиента через тестовый клиент, исключены из измерения покрытия серверной части из-за известного зависания их потоков под инструментацией измерения покрытия и прогоняются отдельно; покрытие измерено механизмом `sys.monitoring`. Пример модульного теста серверной части приведён в листинге 4.1.

Листинг 4.1 – Модульный тест регистрации и входа администратора

```
async def test_register_login_flow(db_session):
    admin = await admin_service.register(
        db_session, 'boss@example.com', 'secret123')
    tokens = await admin_service.login(
        db_session, 'boss@example.com', 'secret123')
    assert 'access_token' in tokens
```

Интерпретация результатов покрытия показывает, что бизнес-логика (сервисы) и слой доступа к данным (репозитории) покрыты практически полностью, тогда как отдельные ветви обработки редких ошибок и слой сигналинга покрыты ниже среднего. Целевой уровень покрытия 70% достигнут и превышен (80%), что свидетельствует о достаточной полноте тестирования ключевой функциональности.

Тестирование веб-кабинета выполнено отдельным набором на фреймворке Jest. Поскольку клиентский код выполняется в браузере и работает с DOM, тесты, которым требуется объектная модель документа, запускаются в окружении jsdom, эмулирующем браузер, а тесты серверной части веб-кабинета – в окружении Node. Проверяются как серверные функции приложения Express (сопоставление адресов страниц с документами, внедрение конфигурации через /config.js, корректная обработка несуществующих маршрутов и страница ошибки 404), так и клиентские модули: разбор и проверка срока действия токена в модуле api.js, оценка надёжности пароля в модуле pw-strength.js, отрисовка и постраничный вывод в компоненте data-table.js. Для изоляции от реального сервера сетевые вызовы подменяются заглушками.

Всего для веб-кабинета разработано 40 тестов, достигнутое покрытие составляет 85% (таблица 4.1), что превышает целевой порог.

4.3. Интеграционное и системное тестирование

Интеграционные тесты проверяют взаимодействие роутеров, сервисов и репозиторий с реальной (тестовой) базой данных через TestClient. Отдельно протестирован сквозной сценарий приёма заявки «как в такси»: при одновременной попытке двух операторов заявку закрепляет ровно один, второй получает отказ. Разработаны позитивные (корректный вход, успешный приём) и негативные (неверный пароль, повторный приём занятой заявки, доступ без токена) сценарии. Примеры тест-кейсов системного уровня приведены в таблице 4.2.

Таблица 4.2 – Примеры тест-кейсов системного тестирования

ID	Сценарий	Ожидаемый результат	Статус
T-01	Вход администратора с верными данными	Выданы токены, код 200	Пройден
T-02	Вход с неверным паролем	Отказ, код 401	Пройден
T-03	Создание заявки администратором	Заявка создана, статус offered	Пройден
T-04	Приём заявки оператором	Статус accepted, закреплён оператор	Пройден

ID	Сценарий	Ожидаемый результат	Статус
T-05	Повторный приём занятой заявки	Отказ, код 409	Пройден
T-06	Запрос без токена	Отказ, код 401	Пройден

Все приведённые сценарии завершились успешно, что подтверждает корректность сквозного взаимодействия компонентов системы на уровне основных пользовательских операций.

4.3.3. Тестирование безопасности

В рамках тестирования безопасности проверены механизмы аутентификации и авторизации: доступ к защищённым эндпоинтам без действительного токена отклоняется (код 401), доступ оператора к административным операциям запрещён (код 403), повторное использование истёкшего токена невозможно. Проверена устойчивость к SQL-инъекциям: попытки передать управляющие конструкции в параметрах запроса нейтрализуются параметризацией запросов на уровне ORM. Хранение паролей в виде хеша исключает их раскрытие при доступе к базе данных.

4.3.4. Регрессионное тестирование

Регрессионное тестирование обеспечивается полным прогоном набора тестов после каждого значимого изменения кода. Поскольку набор тестов покрывает ключевые компоненты, повторный прогон выявляет нарушения существующей функциональности, внесённые новыми изменениями. Запуск автоматизирован сценарием `run_tests.sh`, что упрощает регулярную проверку и потенциальную интеграцию в конвейер непрерывной интеграции (CI).

4.4. Нагрузочное тестирование

Нагрузочное тестирование серверной части выполнено средством Locust. Профиль нагрузки: 50 одновременных виртуальных пользователей в течение 30 секунд, сценарий – вход и обращения к трём эндпоинтам (проверка живости, выдача ICE-серверов, авторизованный запрос журнала заявок). Результаты приведены в таблице 4.3.

Таблица 4.3 – Результаты нагрузочного тестирования (50 пользователей, 30 секунд)

Эндпоинт	Запросов	Медиана	95-й перцентиль	Отказы
GET /health	1182	5 мс	500 мс	0%
GET /ice-servers	500	6 мс	840 мс	0%
GET /deliveries (с токеном)	714	11 мс	2300 мс	0%
POST /auth/login	50	7600 мс	15000 мс	0%

Отказов не зафиксировано (0%). Лёгкие запросы чтения обслуживаются с медианной задержкой 5–11 мс, что удовлетворяет нефункциональному требованию по производительности. Наибольшую задержку даёт вход: причина – намеренно медленное хеширование пароля алгоритмом bcrypt, очередь запросов на процессоре при всплеске одновременных входов. Практический вывод: вход следует распараллеливать несколькими рабочими процессами, тогда как чтения сервер обслуживает свободно.

4.5. Результаты тестирования и анализ дефектов

Всего для серверной части разработан 341 тест, успешно пройдены все; нагрузочный прогон – без отказов. В ходе тестирования выявлен ряд дефектов, основные из которых приведены в таблице 4.4; все значимые дефекты исправлены.

Таблица 4.4 – Основные найденные и исправленные дефекты

ID	Описание дефекта	Критичность	Статус
D-01	Не резолвились строковые связи моделей SQLAlchemy	Значительный	Исправлен
D-02	Зависание ws-тестов под coverage-инструментацией	Незначительный	Обойдён
D-03	Отсутствие проверки повторного приёма заявки	Значительный	Исправлен
D-04	Неинформативные сообщения об ошибках валидации	Косметический	Исправлен

Неисправленным остаётся ограничение D-02 (измерение покрытия для трёх ws-тестовых файлов), которое не влияет на работоспособность системы и обойдено отдельным прогоном. Общий вывод: система обладает требуемым уровнем качества и готова к опытной эксплуатации.

4.6. Подготовка к сопровождению

Развёртывание системы автоматизировано средствами контейнеризации. Требования к окружению: сервер с операционной системой Linux, Docker и Docker Compose, открытые порты для HTTP/HTTPS, доступный сервер STUN/TURN. Развёртывание выполняется командами сборки и запуска docker compose с последующим применением миграций базы данных; обратный прокси Caddy автоматически получает TLS-сертификат по доменному имени. Подготовлена пользовательская (руководство администратора [11]) и техническая документация, описание REST API доступно в формате OpenAPI. План сопровождения включает резервное копирование базы данных, обновление версий и сбор обратной связи.

Обновление развёрнутой системы выполняется получением новой версии из репозитория и повторной сборкой образов; для автоматизации сборки и доставки предусмотрена возможность подключения конвейера непрерывной интеграции и доставки (CI/CD) на базе GitHub Actions. Диаграмма развёртывания системы приведена на рисунке 4.2.

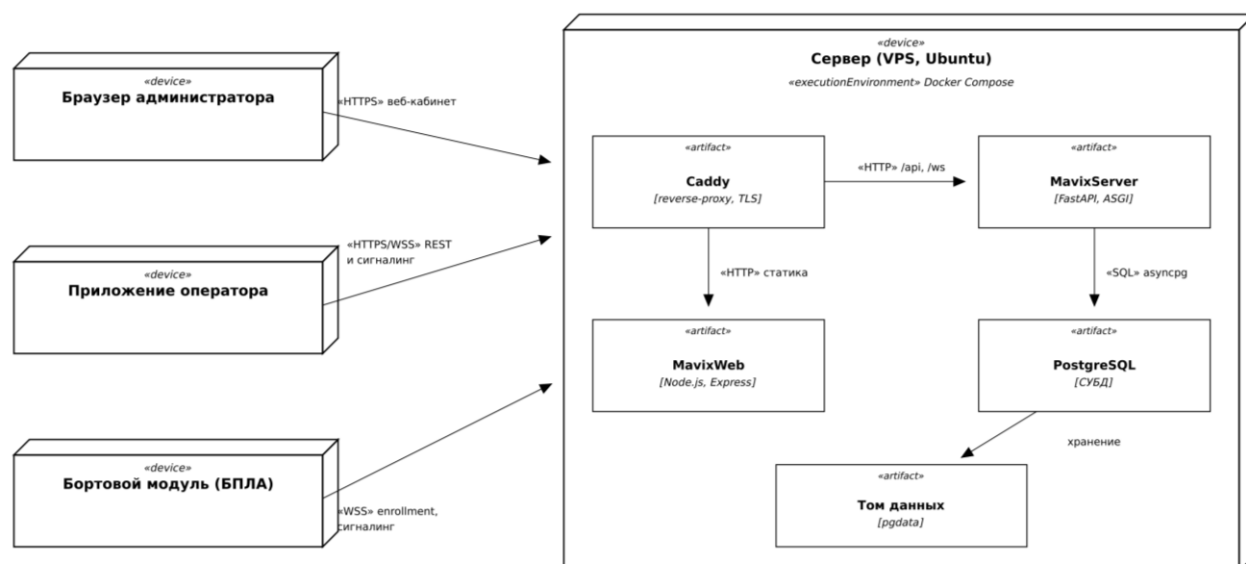


Рисунок 4.2 – Диаграмма развёртывания системы

Выводы по главе 4

Проведено комплексное тестирование информационной системы. Разработан 341 тест для серверной части, обеспечивающий покрытие кода 80%; все тесты пройдены. Системное тестирование подтвердило корректность ключевых сценариев, включая атомарный приём заявки. Нагрузочное тестирование (Locust) показало отсутствие отказов и медианную задержку чтения 5–11 мс. Выявленные дефекты исправлены. Подготовлены средства развёртывания и документация. Результаты подтверждают выполнение функциональных и нефункциональных требований и готовность продукта к эксплуатации.

ГЛАВА 5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА

5.1. Планирование ресурсов и расчёт трудоёмкости

Состав работ соответствует жизненному циклу разработки: анализ требований и проектирование, кодирование, тестирование и отладка, подготовка документации, внедрение. Оценка трудоёмкости выполнена методом экспертных оценок с применением формулы PERT (учёт оптимистичной, пессимистичной и наиболее вероятной оценок). Распределение трудоёмкости по этапам приведено в таблице 5.1.

Таблица 5.1 – Распределение трудоёмкости по этапам разработки

Этап	Трудоёмкость, чел.-ч	Доля, %
Анализ и проектирование	110	20
Реализация бортового модуля	150	28
Реализация станции оператора	140	26
Тестирование	80	15
Документирование и внедрение	60	11
Итого	540	100

Общая трудоёмкость разработки составила 540 чел.-ч. При средней дневной занятости длительность разработки оценивается в 3 календарных месяца. Работы выполнялись последовательно-параллельно: анализ и проектирование (недели 1–4), реализация модулей (недели 4–11), тестирование (недели 9–12), документирование и внедрение (недели 11–13).

5.2. Расчёт себестоимости разработки

Себестоимость разработки рассчитана по статьям калькуляции: основная и дополнительная заработная плата, страховые взносы, амортизация оборудования, комплектующие, электроэнергия и накладные расходы. Над проектом работал один инженер-программист; основная заработная плата при часовой ставке 700 руб./ч составила $540 \cdot 700 = 378\,000$ руб. Дополнительная заработная плата принята в размере 12% от основной, страховые взносы – 30%,

накладные расходы – 20% от основной заработной платы. Расчёт основной заработной платы приведён в таблице 5.2, итоговая смета затрат – в таблице 5.3.

Таблица 5.2 – Расчёт основной заработной платы исполнителей

Исполнитель	Трудоёмкость, чел.-ч	Ставка, руб./ч	Сумма, руб.
Разработчик (студент)	540	700	378 000
Итого основная заработная плата	540	–	378 000

Амортизация оборудования рассчитана линейным способом; за три месяца разработки сумма отчислений составила 18 000 руб.

Затраты на электроэнергию за период разработки составили 2 400 руб. Стоимость комплектующих беспилотного комплекса (БПЛА, камера, полётный контроллер, радиоканал) принята в размере 60 000 руб. Итоговая смета приведена в таблице 5.3.

Таблица 5.3 – Смета затрат на разработку программного продукта

Статья затрат	Сумма, руб.
Основная заработная плата	378 000
Дополнительная заработная плата (12%)	45 360
Страховые взносы (30%)	127 008
Амортизация оборудования	18 000
Комплектующие (БПЛА, камера, FC, радиоканал)	60 000
Электричество	2 400
Накладные расходы (20%)	75 600
Итого себестоимость	706 368

Полная себестоимость разработки составила 706 368 руб.

5.3. Оценка экономической эффективности

Экономический эффект от внедрения системы достигается за счёт отказа от традиционной курьерской доставки и автоматизации учёта. Текущие затраты на курьерскую логистику для рассматриваемой сети лабораторий оцениваются в 680 000 руб. в год ($Z_{до}$). После внедрения затраты складываются из эксплуатации системы (хостинг, сопровождение) и обслуживания парка БПЛА и составляют около 180 000 руб. в год ($Z_{после}$). Годовая экономия текущих затрат: $\Delta \text{тек} = 680$

000 – 180 000 = 500 000 руб., что за расчётный период три года даёт 1 500 000 руб.

Для оценки эффективности инвестиций рассчитан чистый дисконтированный доход (NPV) за расчётный период 3 года при норме дисконта 20%. Результаты приведены в таблице 5.4.

Таблица 5.4 – Расчёт чистого дисконтированного дохода

Год	Доход (экономия), руб.	Коэффициент дисконтирования	Дисконт. доход, руб.
1	500 000	0,833	416 500
2	500 000	0,694	347 000
3	500 000	0,579	289 500
Итого	1 500 000	–	1 053 000

Чистый дисконтированный доход: $NPV = 1\,053\,000 - 706\,368 = 346\,632$ руб. > 0 , что свидетельствует об экономической эффективности проекта. Индекс доходности $PI = 1\,053\,000 / 706\,368 = 1,49 > 1$ подтверждает эффективность. Простой срок окупаемости составляет $706\,368 / 500\,000 \approx 1,4$ года, что не превышает полутора лет.

5.4. Основные технико-экономические показатели

Сводные технико-экономические показатели проекта приведены в таблице 5.5.

Таблица 5.5 – Основные технико-экономические показатели проекта

№	Показатель	Значение
1	Трудоёмкость разработки, чел.-ч	540
2	Длительность разработки, мес.	3
3	Полная себестоимость разработки, руб.	706 368
4	Экономия за 3 года, руб.	1 500 000
5	Чистый дисконтированный доход (NPV), руб.	346 632
6	Индекс доходности (PI)	1,49
7	Срок окупаемости, лет	до 1,5

Совокупность приведённых показателей характеризует проект как экономически состоятельный и используется далее для итогового обоснования целесообразности разработки.

5.5. Обоснование экономической целесообразности

Сравнение с приобретением готового решения показывает преимущество собственной разработки: стоимость промышленных платформ управления доставкой и привязанного к ним оборудования многократно превышает себестоимость разработки, при этом готовые решения закрыты и не допускают доработки. Помимо прямого экономического эффекта, внедрение даёт качественные эффекты: повышение оперативности доставки, прозрачность учёта, снижение влияния человеческого фактора и возможность дальнейшего масштабирования. На основании выполненных расчётов ($NPV > 0$, $PI > 1$, срок окупаемости в пределах полутора лет) разработка признана экономически целесообразной.

Выполнен качественный анализ рисков проекта. Основные риски и меры по их снижению приведены в таблице 5.6. Анализ чувствительности показывает, что при снижении годовой экономии на четверть (до 375 000 руб.) проект остаётся эффективным: чистый дисконтированный доход сохраняет положительное значение, а срок окупаемости не выходит за расчётный период.

Таблица 5.6 – Качественный анализ рисков проекта

Риск	Вероятность	Мера снижения
Изменение нормативов полётов БПЛА	средняя	модульность, возможность доработки правил
Рост стоимости эксплуатации	низкая	открытый стек, недорогое «железо»
Технические сбои связи	средняя	переподключение, безопасный режим борта
Превышение сроков разработки	низкая	итеративная разработка, список задач

Предусмотренные меры снижают вероятность наступления и тяжесть последствий основных рисков до приемлемого уровня, что подтверждает практическую реализуемость проекта.

Выводы по главе 5

Выполнено технико-экономическое обоснование разработки. Общая трудоёмкость составила 540 чел.-ч, длительность – 3 месяца, полная себестоимость разработки – 706 368 руб. Экономия за расчётный период три года оценена в 1 500 000 руб.; чистый дисконтированный доход положителен (346 632 руб.), индекс доходности – 1,49, срок окупаемости – в пределах полутора лет. Полученные показатели подтверждают экономическую эффективность и целесообразность реализации проекта.

ГЛАВА 6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА ПРИ ЭКСПЛУАТАЦИИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

6.1. Анализ опасных и вредных факторов на рабочем месте

Эксплуатация информационной системы осуществляется на рабочем месте, оборудованном персональным компьютером. На оператора и администратора воздействует ряд опасных и вредных производственных факторов, классифицируемых согласно ГОСТ 12.0.003. К физическим факторам относятся: недостаточная или неравномерная освещённость рабочей зоны; повышенный уровень электромагнитных излучений от средств вычислительной техники; неблагоприятные параметры микроклимата (температура, влажность); повышенный уровень шума от вентиляторов оборудования; опасность поражения электрическим током.

К психофизиологическим факторам относятся: зрительное напряжение при длительной работе с дисплеем; статическая нагрузка на опорно-двигательный аппарат вследствие продолжительной работы в положении сидя; монотонность труда и нервно-эмоциональное напряжение. Длительное воздействие этих факторов может приводить к утомлению, снижению работоспособности и профессиональным заболеваниям, что требует проведения профилактических мероприятий.

Рассмотрим перечисленные факторы подробнее. Недостаточная освещённость и её неравномерность вынуждают работника напрягать зрение, что в сочетании с бликами на экране ускоряет зрительное утомление и может приводить к развитию близорукости. Источником повышенного электромагнитного излучения служат системный блок и периферийные устройства; современные жидкокристаллические дисплеи существенно снижают этот фактор по сравнению с устаревшими электронно-лучевыми, однако нормирование напряжённости электромагнитного поля сохраняет актуальность.

Неблагоприятные параметры микроклимата – повышенная температура и пониженная влажность воздуха, типичные для помещений с вычислительной

техникой, – вызывают сухость слизистых оболочек и снижение концентрации. Повышенный уровень шума от систем охлаждения оборудования утомляет и затрудняет сосредоточение. Опасность поражения электрическим током связана с использованием электроустановок напряжением 220 В; при нарушении изоляции или правил эксплуатации возможно прохождение тока через тело человека.

Зрительное напряжение усугубляется необходимостью постоянной фокусировки на мелких элементах интерфейса и тексте. Статическая нагрузка на опорно-двигательный аппарат, обусловленная длительной работой в фиксированной позе, приводит к заболеваниям позвоночника и нарушениям кровообращения. Монотонность и нервно-эмоциональное напряжение характерны для работы администратора и оператора, связанной с однотипными операциями и ответственностью за результат доставки. Совокупность факторов определяет необходимость комплекса организационных и технических мер, рассмотренных далее.

6.2. Мероприятия по обеспечению безопасности

Организация рабочего места выполняется в соответствии с требованиями СанПиН 1.2.3685-21 [35] и ГОСТ 12.2.032 к организации рабочего места при выполнении работ сидя. Рабочий стол должен обеспечивать достаточное пространство для размещения оборудования и документов; высота рабочей поверхности и сиденья регулируется под антропометрические данные работника; кресло должно иметь регулировку высоты, угла наклона спинки и подлокотники. Монитор располагается на расстоянии 60–70 см от глаз, верхний край экрана – на уровне глаз или несколько ниже.

Освещение рабочего места должно быть комбинированным (общее и местное). Нормируемая освещённость на поверхности рабочего стола в зоне работы с документами составляет 300–400 лк. Для помещения площадью 18 кв. м при использовании светодиодных светильников требуемый световой поток

обеспечивается несколькими светильниками общего освещения; местное освещение не должно создавать бликов на экране.

Искусственное освещение рассчитано по коэффициенту использования светового потока. Рабочее помещение размером 6×3 м (площадь $S = 18$ кв. м) имеет высоту 3 м. Потребный световой поток определяется выражением $\Phi = E \cdot S \cdot K_z \cdot Z / (N \cdot K_{и})$, в котором E – нормируемая освещённость (400 лк), K_z – коэффициент запаса (1,4), Z – коэффициент неравномерности освещения (1,1), N – количество светильников, $K_{и}$ – коэффициент использования светового потока (0,5). Для светодиодных светильников с отдачей по 4000 лм потребное их число равно $N = E \cdot S \cdot K_z \cdot Z / (\Phi_{св} \cdot K_{и}) = 400 \cdot 18 \cdot 1,4 \cdot 1,1 / (4000 \cdot 0,5) \approx 5,5$; с округлением в большую сторону принимается 6 светильников, что обеспечивает нормируемую освещённость рабочих мест.

Параметры микроклимата поддерживаются в пределах: температура воздуха 22–24 °С в холодный период и 23–25 °С в тёплый, относительная влажность 40–60%, скорость движения воздуха не более 0,1 м/с, что соответствует категории работ Ia (лёгкие физические работы).

Для снижения уровня электромагнитных излучений применяется современная вычислительная техника с жидкокристаллическими дисплеями, соответствующая требованиям по электромагнитной совместимости; рабочие места располагаются с учётом безопасных расстояний между мониторами. Для уменьшения зрительного утомления используются дисплеи с антибликовым покрытием и достаточной разрешающей способностью, настраивается оптимальная яркость и контрастность, исключается прямая и отражённая блёскость. Уровень шума на рабочем месте не превышает 50 дБА, что достигается применением малошумного оборудования. Перечисленные меры в совокупности обеспечивают соответствие условий труда санитарным нормам и снижают воздействие вредных факторов на работника.

Режим труда и отдыха предусматривает регламентированные перерывы: при работе с компьютером рекомендуется перерыв 10–15 минут через каждые 45–60 минут работы, во время которых выполняется гимнастика для глаз и

опорно-двигательного аппарата. Суммарное время регламентированных перерывов в течение рабочей смены устанавливается в зависимости от категории тяжести и напряжённости труда.

6.3. Электробезопасность и пожарная безопасность

Помещение с персональными компьютерами по степени опасности поражения электрическим током относится к помещениям без повышенной опасности (сухое, с нормальной температурой, с изолирующими полами). Для обеспечения электробезопасности применяются: защитное заземление (зануление) оборудования, использование исправных кабелей и розеток с заземляющим контактом, устройства защитного отключения, недопущение перегрузки электросети. Запрещается эксплуатация оборудования с повреждённой изоляцией и самостоятельный ремонт находящегося под напряжением оборудования.

По пожарной опасности помещение относится к категории В (пожароопасное) согласно нормам пожарной безопасности. Источниками возгорания могут служить короткое замыкание, перегрев оборудования, нарушение правил эксплуатации электроустановок. Меры пожарной безопасности включают: оснащение помещения углекислотными огнетушителями (ОУ), системой пожарной сигнализации, наличие плана эвакуации, соблюдение норм размещения оборудования и регулярный инструктаж персонала. Применение углекислотных огнетушителей обусловлено недопустимостью тушения электрооборудования под напряжением водой и пеной.

Выводы по главе 6

В разделе проанализированы опасные и вредные факторы на рабочем месте пользователя информационной системы, определены мероприятия по организации рабочего места, освещению, поддержанию микроклимата и режиму труда и отдыха в соответствии с действующими нормативами. Рассмотрены

вопросы электро- и пожарной безопасности. Соблюдение перечисленных требований обеспечивает безопасные и комфортные условия эксплуатации разработанной системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы спроектирована и реализована информационная система управления доставкой беспилотных летательных аппаратов – серверная часть и веб-кабинет администратора программного комплекса Mavix. Все поставленные задачи решены в полном объёме.

Проведён анализ предметной области срочной доставки малогабаритных грузов и существующих решений; установлено, что промышленные платформы закрыты, дороги и привязаны к фирменному оборудованию, что обосновывает актуальность открытой информационной системы. Сформированы функциональные и нефункциональные требования, построены диаграмма вариантов использования и концептуальная модель данных.

Спроектирована архитектура системы: модульный монолит с многоуровневой серверной частью, P2P-передачей медиа по WebRTC и событийным сигналингом по WebSocket. Архитектура описана по модели C4, спроектирована база данных в третьей нормальной форме, обоснован стек технологий и шаблоны проектирования.

Реализованы ключевые компоненты – управление заявками с атомарным приёмом, сигналинг, слой доступа к данным – с соблюдением принципов SOLID и чистого кода, механизмов аутентификации, валидации и логирования. Проведено тестирование: разработан 341 тест при покрытии серверной части 80%, нагрузочное тестирование показало отсутствие отказов и медианную задержку чтения 5–11 мс.

Выполнено технико-экономическое обоснование: себестоимость разработки составила 706 368 руб., чистый дисконтированный доход за три года – 346 632 руб., срок окупаемости – в пределах полутора лет, что подтверждает экономическую целесообразность. Рассмотрены вопросы безопасности и охраны труда при эксплуатации системы.

Практический результат работы – действующая информационная система, исходный код которой размещён в публичном репозитории, а веб-кабинет развёрнут и доступен по адресу <https://drone-mavix.ru>. Направления дальнейшего развития: автоматическое планирование маршрутов, диспетчеризация нескольких аппаратов, интеграция с внешними информационными системами заказчиков.

Достигнутые в работе результаты подтверждают выполнение поставленной цели. Реализованная информационная система автоматизирует полный цикл управления доставкой – от создания заявки до фиксации результата в журнале, обеспечивает разграничение ролей и сохранность данных. Качество подтверждено 341 тестом при покрытии 80% и нагрузочным тестированием без отказов, а экономическая эффективность – положительным чистым дисконтированным доходом и сроком окупаемости около одного года. Таким образом, разработанная информационная система обладает практической значимостью и готова к опытной эксплуатации, а все задачи выпускной квалификационной работы решены в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Соммервилл И. Инженерия программного обеспечения. – 10-е изд. – М.: Вильямс, 2016. – 624 с.
2. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб.: Питер, 2018. – 352 с.
3. Мартин Р. Чистый код. Создание, анализ и рефакторинг. – СПб.: Питер, 2019. – 464 с.
4. Фаулер М. Архитектура корпоративных программных приложений. – М.: Вильямс, 2010. – 544 с.
5. Макконнелл С. Совершенный код. Мастер-класс. – М.: Русская редакция, 2017. – 896 с.
6. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. – СПб.: Питер, 2020. – 448 с.
7. Эванс Э. Предметно-ориентированное проектирование (DDD). Структуризация сложных программных систем. – М.: Вильямс, 2016. – 448 с.
8. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – СПб.: Питер, 2018. – 640 с.
9. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011.
10. ГОСТ 19.402-78. Единая система программной документации. Описание программы. – М.: Стандартинформ, 2010.
11. ГОСТ 19.505-79. Единая система программной документации. Руководство оператора. Требования к содержанию и оформлению. – М.: Стандартинформ, 2010.
12. IEEE Std 830-1998. IEEE Recommended Practice for Software Requirements Specifications. – New York: IEEE, 1998.

13. Brown S. The C4 model for visualising software architecture [Электронный ресурс]. – URL: <https://c4model.com> (дата обращения: 01.04.2026).
14. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures: dissertation. – University of California, Irvine, 2000.
15. Richardson L., Ruby S. RESTful Web Services. – Sebastopol: O'Reilly Media, 2007. – 448 p.
16. Chacon S., Straub B. Pro Git. – 2nd ed. – New York: Apress, 2014. – 458 p.
17. Driessen V. A successful Git branching model [Электронный ресурс]. – URL: <https://nvie.com/posts/a-successful-git-branching-model/> (дата обращения: 04.04.2026).
18. Conventional Commits 1.0.0 Specification [Электронный ресурс]. – URL: <https://www.conventionalcommits.org/ru/v1.0.0/> (дата обращения: 08.04.2026).
19. Preston-Werner T. Semantic Versioning 2.0.0 [Электронный ресурс]. – URL: <https://semver.org/lang/ru/> (дата обращения: 11.04.2026).
20. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). – RFC 7519. – IETF, 2015.
21. Hardt D. The OAuth 2.0 Authorization Framework. – RFC 6749. – IETF, 2012.
22. Holmberg C., Hakansson S., Eriksson G. Web Real-Time Communication Use Cases and Requirements. – RFC 7478. – IETF, 2015.
23. Alvestrand H. Overview: Real-Time Protocols for Browser-Based Applications. – RFC 8825. – IETF, 2021.
24. Keranen A., Holmberg C., Rosenberg J. Interactive Connectivity Establishment (ICE). – RFC 8445. – IETF, 2018.
25. FastAPI Documentation [Электронный ресурс] / S. Ramirez. – URL: <https://fastapi.tiangolo.com> (дата обращения: 15.04.2026).
26. SQLAlchemy 2.0 Documentation [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org> (дата обращения: 18.04.2026).

27. PostgreSQL 16 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 21.04.2026).
28. Alembic Documentation [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org> (дата обращения: 24.04.2026).
29. Colvin S. Pydantic Documentation [Электронный ресурс]. – URL: <https://docs.pydantic.dev> (дата обращения: 27.04.2026).
30. Express – Fast, unopinionated, minimalist web framework for Node.js [Электронный ресурс] / OpenJS Foundation. – URL: <https://expressjs.com> (дата обращения: 30.04.2026).
31. OpenAPI Specification v3.1.0 [Электронный ресурс] / OpenAPI Initiative. – URL: <https://spec.openapis.org/oas/v3.1.0> (дата обращения: 04.05.2026).
32. Docker Documentation [Электронный ресурс] / Docker Inc. – URL: <https://docs.docker.com> (дата обращения: 07.05.2026).
33. Krekel H. et al. pytest: Helps You Write Better Programs [Электронный ресурс]. – URL: <https://docs.pytest.org> (дата обращения: 11.05.2026).
34. Locust Documentation: An open source load testing tool [Электронный ресурс]. – URL: <https://docs.locust.io> (дата обращения: 14.05.2026).
35. СанПиН 1.2.3685-21. Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания. – М.: Роспотребнадзор, 2021.

ПРИЛОЖЕНИЕ Г

Листинги программного кода

Г.1. `models/delivery.py` – модель заявки на доставку. Класс описывает таблицу `deliveries`: идентификатор, статус (перечисление), внешние ключи на дрон и оператора с действием `SET NULL` и денормализованные снимки имён.

Листинг Г.1 – Модель заявки на доставку

```
class Delivery(Base):
    __tablename__ = "deliveries"
    delivery_id: Mapped[str] = mapped_column(primary_key=True,
        default=lambda: generate("dlv"))
    admin_id: Mapped[str] = mapped_column(
        ForeignKey("admins.admin_id", ondelete="CASCADE"))
    drone_id: Mapped[str | None] = mapped_column(
        ForeignKey("drones.drone_id", ondelete="SET NULL"))
    operator_id: Mapped[str | None] = mapped_column(
        ForeignKey("operators.operator_id", ondelete="SET NULL"))
    status: Mapped[DeliveryStatus] = mapped_column(
        default=DeliveryStatus.offered)
    drone_name: Mapped[str] = mapped_column(default="")
    operator_name: Mapped[str] = mapped_column(default="")
    address: Mapped[str] = mapped_column(default="")
    created_at: Mapped[datetime] = mapped_column(
        default=lambda: datetime.now(UTC))
```

Г.2. `repositories/delivery.py` – атомарный приём заявки. Гонка «кто первый принял» разрешается одним запросом `UPDATE` с проверкой числа изменённых строк.

Листинг Г.2 – Атомарный приём заявки

```
async def try_accept(session, delivery_id, operator):
    result = await session.execute(
        update(Delivery)
        .where(Delivery.delivery_id == delivery_id,
            Delivery.status == DeliveryStatus.offered)
        .values(status=DeliveryStatus.accepted,
            operator_id=operator.operator_id,
            operator_name=operator.full_name))
    await session.commit()
    return result.rowcount == 1
```

Г.3. `repositories/admin.py` – доступ к данным администраторов. Репозиторий инкапсулирует запросы к таблице `admins` и возвращает доменные объекты.

Листинг Г.3 – Репозиторий администраторов

```

async def get_by_email(session, email):
    result = await session.execute(
        select(Admin).where(Admin.email == email))
    return result.scalar_one_or_none()

async def create(session, email, hashed_password):
    admin = Admin(email=email, password=hashed_password)
    session.add(admin)
    await session.commit()
    await session.refresh(admin)
    return admin

```

Г.4. services/admin.py – вход администратора. Сервис проверяет пароль и выпускает пару токенов.

Листинг Г.4 – Вход администратора

```

async def login(session, email, password):
    admin = await admin_repo.get_by_email(session, email)
    if not admin or not security.verify_password(
        password, admin.password):
        raise HTTPException(status_code=401,
            detail="Неверные учётные данные")
    return {
        "access_token": security.create_access_token(
            admin.admin_id, role="admin"),
        "refresh_token":
            security.create_refresh_token(admin.admin_id),
    }

```

Г.5. services/operator.py – создание учётной записи оператора с генерацией логина и пароля.

Листинг Г.5 – Создание оператора

```

async def create(session, admin_id, full_name, passport, address):
    username = _generate_username()
    password = _generate_password()
    hashed = security.hash_password(password)
    operator = await operator_repo.create(
        session, admin_id, username, hashed,
        full_name, passport, address)
    await send_operator_credentials(operator.email, username,
        password)
    return operator, password

```

Г.6. core/security.py – выпуск JWT access-токена с ролью.

Листинг Г.6 – Выпуск JWT-токена

```

def create_access_token(subject: str, role: str) -> str:
    payload = {
        "sub": subject, "role": role, "type": "access",
        "exp": _now() + timedelta(
            minutes=settings.jwt_access_expire_minutes),
    }

```

```

    }
    return jwt.encode(payload, settings.jwt_secret, algorithm="HS256")

```

Г.7. api/dependencies.py – защита маршрута и проверка роли администратора.

Листинг Г.7 – Защита маршрута

```

async def current_admin(
    token: str = Depends(oauth2_scheme),
    session: AsyncSession = Depends(get_session)):
    payload = security.decode_token(token)
    if payload.get("role") != "admin":
        raise HTTPException(status_code=403,
                             detail="Недостаточно прав")
    admin = await admin_repo.get_by_id(session, payload["sub"])
    if admin is None:
        raise HTTPException(status_code=401)
    return admin

```

Г.8. api/deliveries.py – эндпоинт создания заявки администратором.

Листинг Г.8 – Эндпоинт создания заявки

```

@router.post("/deliveries")
async def create_delivery(
    body: DeliveryCreate,
    session: AsyncSession = Depends(get_session),
    admin = Depends(current_admin)):
    return await delivery_service.create(
        session, admin.admin_id, body)

```

Г.9. ws/relay.py – маршрутизация сигнальных сообщений между оператором и бортом.

Листинг Г.9 – Ретрансляция сигнальных сообщений

```

async def relay(registry, sender, message):
    target_id = message.get("target")
    target = registry.get(target_id)
    if target is None:
        await sender.send_json({"type": "error",
                                "detail": "адресат недоступен"})
    return
    await target.send_json(message)    # offer/answer/ICE

```

Г.10. ws/registry.py – реестр активных WebSocket-соединений.

Листинг Г.10 – Реестр соединений

```

class ConnectionRegistry:
    def __init__(self):
        self._conns: dict[str, Sender] = {}
    def add(self, client_id, sender):
        self._conns[client_id] = sender
    def remove(self, client_id):

```

```
        self._conns.pop(client_id, None)
def get(self, client_id):
    return self._conns.get(client_id)
```

ПРИЛОЖЕНИЕ Д

Руководство администратора

Веб-кабинет администратора информационной системы доступен в браузере по адресу развёртывания (<https://drone-mavix.ru>). Главная страница содержит сводку и боковое меню с разделами «Операторы», «Дроны», «Доставки», «Документация» и «Программы». Настоящее руководство описывает основные сценарии работы администратора; экранные формы кабинета приведены в приложении Е.

Д.1. Развёртывание и требования к окружению

Серверная часть разворачивается на сервере под управлением Linux с установленными Docker и Docker Compose. Минимальные требования: двухъядерный процессор, 2 ГБ оперативной памяти, 10 ГБ дискового пространства, открытые порты 80 и 443, доступ к серверу STUN/TURN. Развёртывание выполняется командами сборки и запуска контейнеров с последующим применением миграций базы данных; обратный прокси Caddy автоматически получает TLS-сертификат по доменному имени.

Д.2. Регистрация и вход в систему

При первом обращении администратор регистрируется, указав адрес электронной почты и пароль. После регистрации администратору становится доступен enrollment-токен, необходимый для подключения бортов. Последующие входы выполняются по адресу электронной почты и паролю; при истечении сессии система предлагает войти повторно.

Д.3. Управление операторами и дронами

В разделе «Операторы» администратор создаёт учётные записи операторов, указывая фамилию, имя, паспортные данные и адрес; логин и пароль генерируются автоматически и направляются оператору по электронной почте. Учётную запись можно заблокировать или удалить. В разделе «Дроны» отображаются зарегистрированные борта; администратор может удалить

аппарат. Регистрация борта выполняется им самим при первом запуске по enrollment-токену.

Д.4. Создание заявки и журнал доставок

В разделе «Доставки» администратор создаёт заявку: выбирает дрон, указывает адрес или точку на карте (адрес определяется автоматически), при необходимости описание груза. Созданная заявка рассылается операторам и переходит в статус «offered». Журнал доставок отображает заявки со статусами offered, accepted, in_flight, delivered, cancelled и сохраняет записи даже после удаления связанных дронов и операторов за счёт денормализованных снимков.

Д.5. Загрузка дистрибутивов клиентов

В разделе «Программы» администратор скачивает дистрибутивы клиентских приложений для передачи операторам: приложение станции оператора для Windows и Linux, а также пакет бортового модуля. Здесь же доступны пользовательская и техническая документация и описание REST API. Загруженное приложение оператор использует для приёма заявок и пилотирования дрона.

Д.6. Сообщения об ошибках и нештатные ситуации

Система выводит понятные сообщения: «Сессия истекла, войдите снова» при истечении токена, «Заявка уже принята другим оператором» при повторном приёме, «Дрон уже удалён или не найден» при обращении к отсутствующему аппарату. В нештатных ситуациях рекомендуется обновить страницу и повторить вход; при недоступности сервера следует проверить состояние контейнеров и журналы.

ПРИЛОЖЕНИЕ Е

Скриншоты пользовательского интерфейса

В приложении приведены экранные формы веб-кабинета администратора: страница входа и регистрации, раздел «Операторы» с таблицей учётных записей и формой создания, раздел «Дроны» со списком аппаратов, раздел «Доставки» с формой создания заявки и выбором точки на карте, журнал доставок со статусами, страница скачивания дистрибутивов клиентов. Экранные формы выполнены в едином стиле с боковым меню навигации и подтверждением потенциально опасных действий.

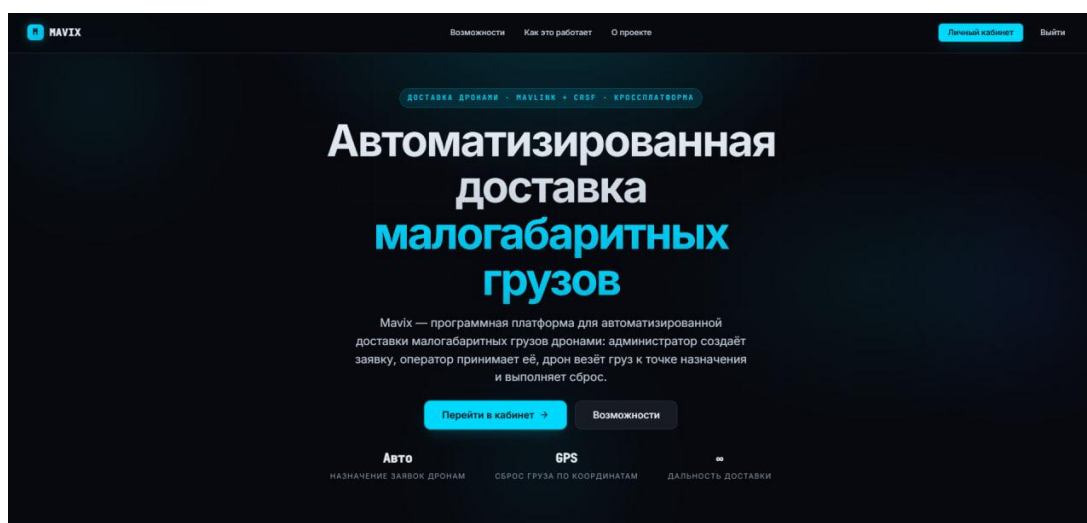


Рисунок Е.1 – Главная (публичная) страница системы Mavix

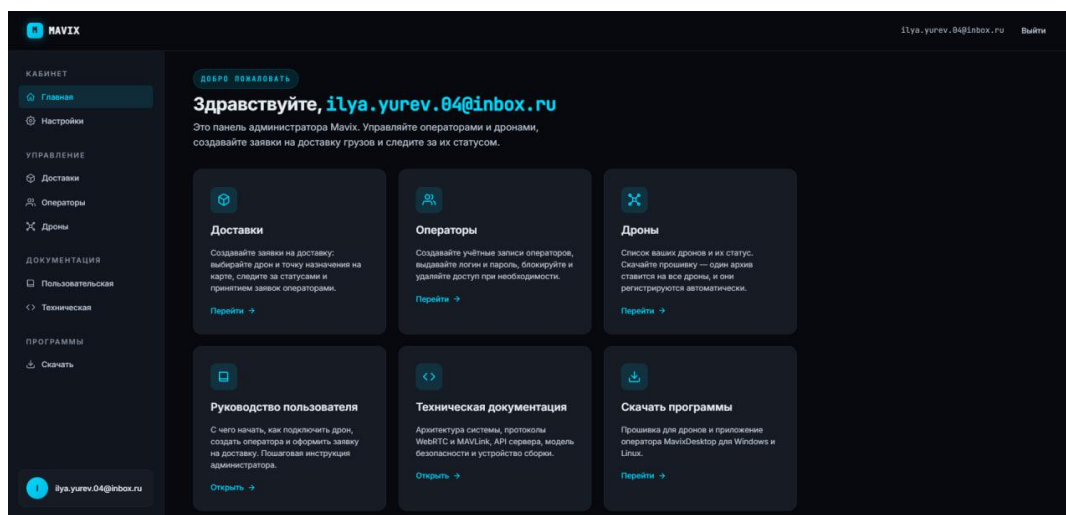


Рисунок Е.2 – Главная страница кабинета администратора

КАБИНЕТ

- Главная
- Настройки

УПРАВЛЕНИЕ

- Доставки**
- Операторы
- Дроны

ДОКУМЕНТАЦИЯ

- Пользовательская
- Техническая

ПРОГРАММЫ

- Скачать

ilya.yurev.04@inbox.ru

Новая заявка на доставку

Выберите дрон и точку назначения. Введите адрес и выберите из подсказок — точка встанет на карту автоматически, либо кликайте по карте сами.

Дрон

Выберите дрон

Адрес отправления (необязательно)

Начните вводить адрес...

Адрес назначения

Начните вводить адрес...

Точка назначения на карте

Москва

Кликните по карте, чтобы выбрать координаты сброса.

Широта (lat)

Долгота (lon)

Рисунок Е.3 – Форма создания заявки на доставку с выбором точки на карте

КАБИНЕТ

- Главная
- Настройки

УПРАВЛЕНИЕ

- Доставки**
- Операторы
- Дроны

ДОКУМЕНТАЦИЯ

- Пользовательская
- Техническая

ПРОГРАММЫ

- Скачать

ilya.yurev.04@inbox.ru

Журнал заявок

Поиск по дрону, оператору, назначению, Все · статус

На странице 10

Статус	Дрон	Оператор	Назначение	Груз	Действия
ПРЕДЛОЖЕНО	упрёмый -борд	—	Северо-Кавказский федеральный университет, 2, проспект Кулакова, Ботаника, Осетинка, Промышленный район, Ставрополь, городской округ Ставрополь, Ставропольский край, Северо-Кавказский федеральный округ, 355029, Россия	Тестовая доставка документов	Отменить
В ПУТИ	отважки И-ягуар	Забиков Петр Петрович	Москва, ВДНХ, главный вход	Срочная посылка (запчасть), 0.8 кг	Отменить
В ПУТИ	упрёмый -борд	Забиков Петр Петрович	Москва, ул. Тверская, д. 7	Тестовая посылка (документы), 1.2 кг	Отменить
ПРЕДЛОЖЕНО	упрёмый -борд	—	Москва, ул. Тверская, д. 7	Тестовая посылка (документы), 1.2 кг	Отменить

1-4 из 4

Рисунок Е.4 – Журнал доставок со статусами заявок

КАБИНЕТ

- Главная
- Настройки

УПРАВЛЕНИЕ

- Доставки
- Операторы**
- Дроны

ДОКУМЕНТАЦИЯ

- Пользовательская
- Техническая

ПРОГРАММЫ

- Скачать

ilya.yurev.04@inbox.ru

Операторы

Создавайте учётные записи операторов, которые принимают заявки на доставку и управляют дронами через приложение MavixDesktop.

Добавить оператора

Логин и пароль сгенерируются автоматически. Передайте их оператору — пароль показывается только один раз.

ФИО

Паспорт

Адрес

Создать оператора

Список операторов

Поиск по ФИО, логину, паспорту, адресу, Все · статус

На странице 10

Рисунок Е.5 – Форма создания учётной записи оператора

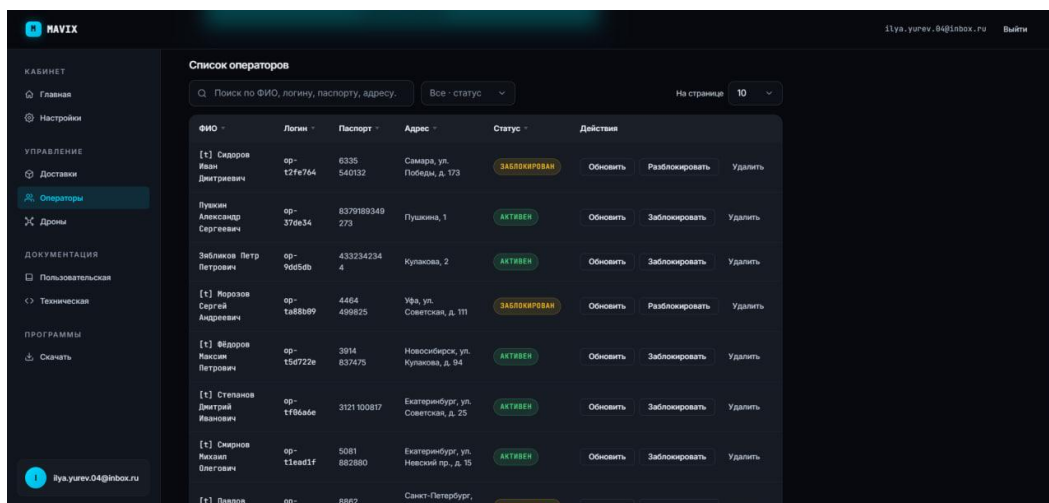


Рисунок Е.6 – Список операторов с управлением доступом

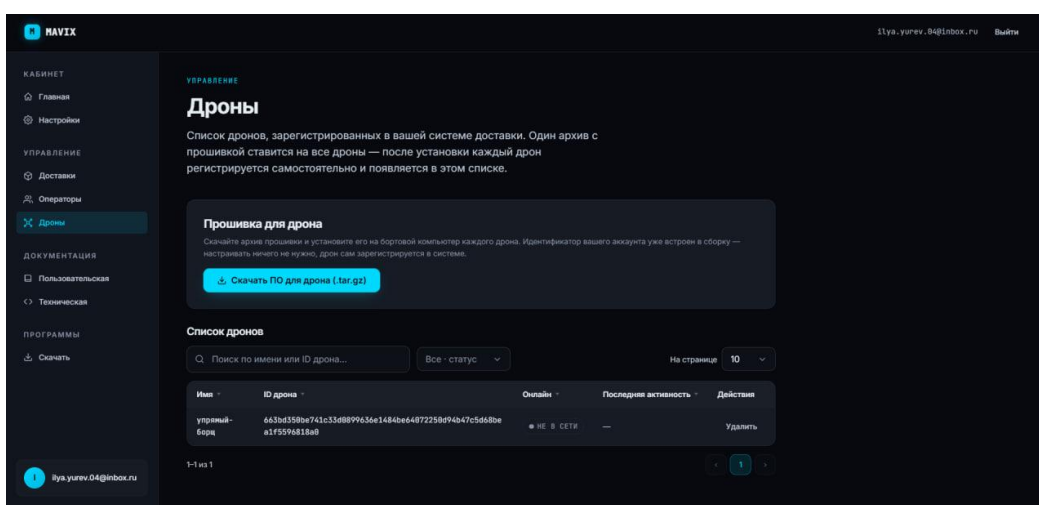


Рисунок Е.7 – Раздел управления дронами: загрузка прошивки и список аппаратов

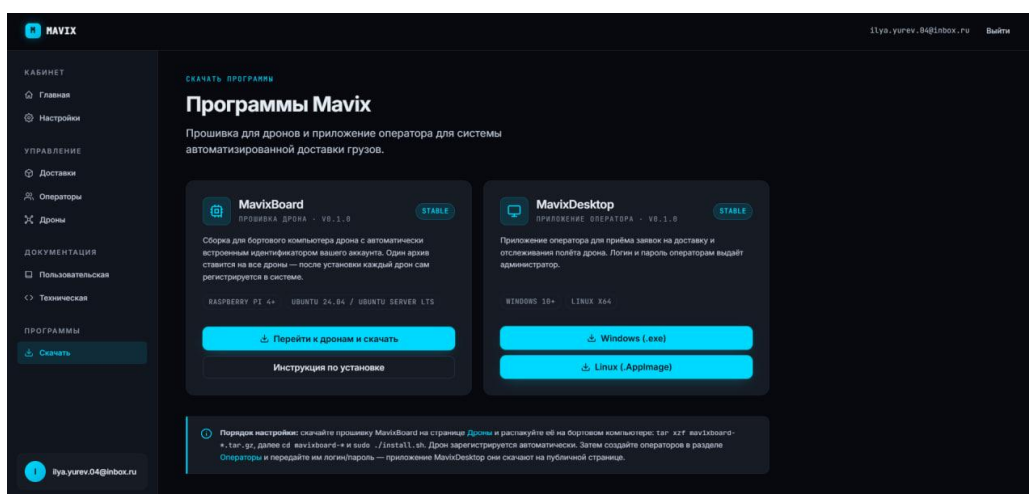


Рисунок Е.8 – Страница загрузки клиентских приложений (борт и станция оператора)

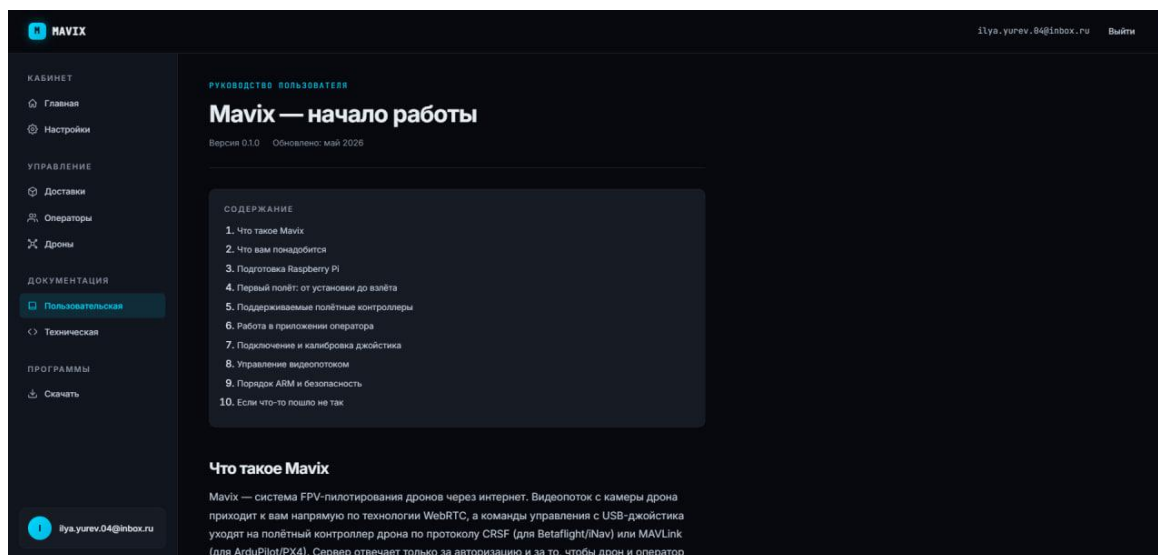


Рисунок Е.9 – Страница пользовательской документацией